

Fondamenti di Telecomunicazioni

Marco Tartaglione

7 ottobre 2026

Indice

1	Introduzione	5
1.1	Classificazione	5
1.2	Wireless	5
1.3	Comunicazione	5
1.3.1	Protocolli	5
1.3.2	Interfaccia	6
1.4	Problemi di progettazione	6
1.5	Servizi	6
1.5.1	Primitive	7
1.6	Modelli	7
1.6.1	ISO-OSI	7
1.6.1.1	Criticità	7
1.6.2	Modello TCP/IP	8
1.6.2.1	Criticità	8
1.6.3	Modello Ibrido	8
1.7	Esempi di reti	9
1.7.1	Architettura di Internet	9
2	Datalink Layer	11
2.1	Ethernet (IEEE 802.3)	11
2.1.1	Frame	11
2.1.2	Indirizzi	11
2.1.3	Collisioni	12
2.1.3.1	CSMA/CD	12
2.1.3.2	Dimensione minima frame	12
2.1.4	Evoluzione	13
2.1.5	Ethernet classica (10 Mbit/s)	13
2.1.6	Fast Ethernet (100 Mbit/s)	13
2.1.7	Gigabit Ethernet (1000 Mbit/s)	14
2.1.8	10-Gigabit Ethernet (10GbE)	15
2.1.9	Dispositivi di commutazione	15
2.2	Wi-Fi (IEEE 802.11)	15
2.2.1	Modalità operative	16
2.2.1.1	Extended Service Set (ESS)	16
2.2.2	Frame	16
2.2.2.1	Dati (type = 10)	16
2.2.2.2	Gestione (type = 00)	17
2.2.2.3	Controllo (type = 01)	17
2.2.3	Condivisione	18
2.2.4	CSMA/CA	18
2.2.5	Stazione nascosta	19
2.2.5.1	RTS/CTS	19
2.2.6	Frammentazione	20
2.2.7	Stazione esposta	21
2.2.7.1	Soluzioni di efficienza	21

3	Network layer	22
3.1	Tipi di indirizzi	22
3.2	Switching	22
3.3	Connessione	22
3.3.1	Creazione Virtual Circuit	23
3.4	Tabelle	24
3.4.1	ARP	24
3.4.2	Routing	24
3.4.3	MTU	24
3.5	Protocolli di livello 3	24
3.5.1	Invio	25
3.5.2	Hop	25
3.5.3	Ricezione	26
3.6	Protocollo IP	26
3.6.1	Versione IPv4	26
3.6.1.1	Storia	26
3.6.1.2	Utilizzo	27
3.6.1.3	Visibilità	27
3.6.1.4	Struttura	27
3.6.2	Versione IPv6	28
3.6.2.1	Notazione	28
3.6.2.2	Struttura	28
3.6.3	Differenze tra gli header IPv4 e IPv6	29
3.7	NAT	29
3.7.1	Processo decisionale	29
3.7.2	PAT	29
3.7.3	Ulteriori utilizzi	30
3.8	Multiplexing	30
4	Transport layer	31
4.1	Servizi	31
4.2	RPC	31
4.3	UDP	31
4.3.1	Servizi	31
4.3.2	Header	32
4.3.3	Utilizzi	32
4.3.4	RTP	32
4.3.4.1	Header	32
4.3.4.2	Jitter	33
4.3.4.3	RTCP	33
4.4	TCP	34
4.4.1	Socket	34
4.4.2	Servizi	34
4.4.3	ACK	34
4.4.3.1	Gestione degli errori	35
4.4.3.2	RTT	35
4.4.4	Segmento	35
4.4.4.1	Timeout	35
4.4.5	Header	35
4.4.5.1	Pseudo Header	36
4.4.6	Connessione	36
4.4.6.1	Connessione Simultanea	37
4.4.6.2	Stati di Connessione	37
4.5	Controllo di Flusso	38

4.5.1	Finestra di Ricezione	39
4.5.2	Timeout e Ritrasmissione	39
4.5.2.1	Calcolo del Timer	39
4.5.2.2	Gestione dei Ritrasmissioni	39
4.5.2.3	Delayed ACK	39
4.5.2.4	Selective ACK	39
4.6	Controllo di Congestione	39
4.6.1	Finestra di Congestione	40
4.6.2	Segnali di Congestione	40
4.6.2.1	ACK Clock	40
4.6.3	Slow Start	40
4.6.4	Congestion Avoidance	41
4.6.5	Fast Retransmission (TCP Tahoe)	41
4.6.6	Fast Recovery (TCP Reno)	42
4.7	Velocità di Trasmissione	42
4.7.1	Perdite	43

1 Introduzione

Le tecnologie di comunicazione più comunemente diffuse sono **broadcast** e **point to point** (anche dette **unicast**). Nelle reti punto-punto i messaggi prendono il nome di **pacchetti** e possono visitare più macchine intermedie prima di arrivare a destinazione. Nelle reti broadcast, invece, ogni pacchetto viene ricevuto da ogni macchina.

1.1 Classificazione

È possibile classificare i tipi di rete sulla base delle loro dimensioni:

- **PAN** (Personal Area Network) 1 m
- **LAN** (Local Area Network) 10 m – 1 km
È solitamente una rete privata e di dimensioni ridotte. I canali di comunicazione possono essere statici o dinamici, oltre che centralizzati o non centralizzati.
- **MAN** (Metropolitan Area Network) 10 km
- **WAN** (Wide Area Network) 100 km – 1000 km
Utilizzano lo stesso principio delle LAN, applicato su larga scala. Non tutti gli host appartengono allo stesso proprietario e le reti connesse tra loro possono utilizzare tecnologie diverse. All'interno delle WAN è possibile che un host sia a sua volta una sottorete.
Si individuano due tipi di WAN:
 - **VPN**: linea dedicata.
 - **Condivise**: la linea è condivisa tra più aziende ed è solitamente fornita da un operatore detto **ISP**.
- **Internet** 10000 km

1.2 Wireless

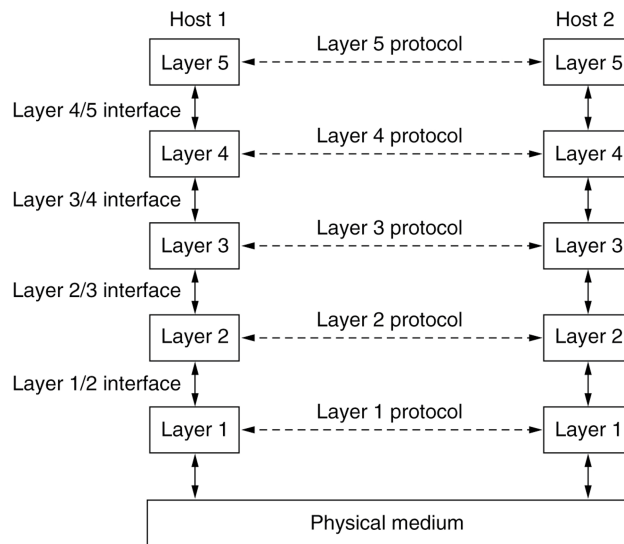
Le reti wireless si dividono in:

- **Interconnesse**: un insieme di reti interconnesse è detto **internetwork** o **internet**.
- **LAN**: ogni computer è dotato di un trasmettitore e di un'antenna per comunicare con gli altri dispositivi. Nella maggior parte dei casi è presente un **access point**, che funge da centro di comunicazione.
- **WAN**: includono sistemi satellitari, reti telefoniche cellulari, ecc.

1.3 Comunicazione

1.3.1 Protocolli

Per ridurre la complessità dei sistemi di comunicazione, molte reti sono organizzate secondo una struttura a strati o **pila di livelli** (**layered architecture**). Ogni livello svolge compiti specifici e comunica esclusivamente con il livello corrispondente della controparte, attraverso un insieme ben definito di regole chiamato **protocollo**. Le entità che operano allo stesso livello su dispositivi diversi sono dette **peer**. L'elenco di tutti i protocolli usati da un sistema è detta **pila di protocolli**.



1.3.2 Interfaccia

La comunicazione tra due livelli adiacenti avviene tramite un insieme di operazioni e servizi messi a disposizione dal livello inferiore, definito come **interfaccia**. Grazie a questa struttura, ogni livello può utilizzare i servizi offerti dal livello sottostante senza preoccuparsi dei dettagli interni della sua implementazione, favorendo modularità e indipendenza tra gli strati.

Durante il percorso dei dati attraverso i vari livelli della pila, ogni livello aggiunge al messaggio un proprio **header** e talvolta anche un **trailer**. Il primo, in particolare, è un'intestazione contenente informazioni di controllo specifiche per il funzionamento del relativo protocollo.

1.4 Problemi di progettazione

La progettazione dei livelli in un'architettura di rete deve affrontare una serie di problematiche fondamentali, indipendenti dalla tecnologia specifica. I principali aspetti da considerare sono:

- **Controllo degli errori:** meccanismi per rilevare e correggere eventuali errori nei dati trasmessi, causati da rumore o interferenze.
- **Routing:** definizione dei percorsi ottimali che i pacchetti devono seguire all'interno della rete, specialmente nelle reti complesse o geograficamente distribuite.
- **Addressing e Naming:** assegnazione di identificatori univoci (indirizzi o nomi) ai dispositivi o ai processi che partecipano alla comunicazione, al fine di consentire il corretto instradamento e l'identificazione dei destinatari.
- **Multiplexing:** condivisione delle risorse di comunicazione tra più flussi di dati o utenti, ottimizzando l'utilizzo della banda disponibile.
- **Controllo del flusso:** regolazione della velocità di trasmissione tra mittente e destinatario, per evitare che un dispositivo più lento venga sovraccaricato di dati.

1.5 Servizi

Ogni livello può offrire principalmente due tipi di servizi alla comunicazione:

- **Servizi orientati alla connessione:** è necessaria la creazione preliminare di una connessione tra mittente e destinatario, che rimane attiva per tutta la durata dello scambio dati. Una volta terminata la comunicazione, la connessione viene chiusa. Questi servizi garantiscono affidabilità, ordine e controllo nella trasmissione. Esempio tipico è il protocollo TCP.

- **Servizi senza connessione:** non viene instaurata alcuna connessione. Ogni pacchetto (detto **data-gram**) viaggia in modo indipendente e include al suo interno l'indirizzo del destinatario. I pacchetti possono seguire percorsi differenti e arrivare in ordine diverso da quello di partenza, o subire perdite. I nodi intermedi adottano la tecnica di **store-and-forward**, ricevendo e memorizzando completamente ciascun pacchetto prima di inoltrarlo verso la tappa successiva. Un esempio è il protocollo UDP.

Inoltre, i servizi possono essere classificati in base al livello di **affidabilità**:

- **Servizi affidabili:** garantiscono che i dati arrivino correttamente a destinazione e nell'ordine previsto. La conferma di ricezione avviene tramite meccanismi come gli **ACK** (acknowledgement) e, in caso di errore o perdita, vengono effettuate ritrasmissioni.
- **Servizi non affidabili:** i pacchetti vengono inviati senza alcuna conferma di ricezione. Non è garantita né la consegna né l'ordine dei dati, ma si ottiene una comunicazione più rapida e leggera.
- **Servizi request-reply (richiesta-risposta):** si basano sull'invio di un singolo datagram di richiesta seguito da un datagram di risposta. Questo modello è comunemente utilizzato in applicazioni come il DNS.

1.5.1 Primitive

Si definiscono **primitive** tutte le operazioni che un servizio mette a disposizione dei livelli superiori. Le primitive rappresentano l'interfaccia tra due livelli della pila protocollare e consentono la gestione della comunicazione in maniera astratta, senza che il livello superiore debba conoscere i dettagli implementativi del livello inferiore.

1.6 Modelli

1.6.1 ISO-OSI

Il modello **ISO-OSI** (International Standard Organization - Open Systems Interconnection) definisce un'architettura di riferimento per la comunicazione tra sistemi aperti, ovvero dispositivi progettati per interagire tra loro all'interno di una rete.

Il modello è suddiviso in **sette livelli**:

- **Applicazione:** fornisce i servizi di rete direttamente alle applicazioni utente.
- **Presentazione:** si occupa della sintassi e della semantica delle informazioni trasmesse, garantendo che i dati siano interpretati correttamente (ad esempio, codifica e compressione).
- **Sessione:** gestisce l'instaurazione, il mantenimento e la terminazione delle sessioni di comunicazione tra sistemi.
- **Trasporto:** assicura il trasferimento affidabile dei dati tra i sistemi terminali, suddividendo le informazioni in **segmenti** e controllandone la corretta consegna.
- **Network:** determina il percorso che i **pacchetti** devono seguire nella rete per raggiungere la destinazione, gestendo l'instradamento e l'indirizzamento.
- **Datalink:** trasforma un collegamento fisico potenzialmente soggetto a errori in una connessione priva di errori, organizzando i dati in **frame**.
- **Fisico:** si occupa della trasmissione dei bit grezzi sul mezzo fisico di comunicazione (cavi, onde radio, fibra ottica, ecc.).

I primi quattro livelli (Applicazione, Presentazione, Sessione e Trasporto) sono presenti solo nei nodi terminali (host), mentre i livelli inferiori (Network, Datalink e Fisico) sono implementati in tutti i dispositivi di rete.

1.6.1.1 Criticità

Nonostante il modello ISO/OSI sia ampiamente utilizzato come riferimento teorico e didattico, ha incontrato diverse difficoltà pratiche nella sua adozione su larga scala. I principali problemi riscontrati sono:

- **Tempismo non adeguato:** il modello è stato formalizzato quando soluzioni alternative, come il modello TCP/IP, erano già consolidate e ampiamente implementate.
- **Tecnologie immature:** alcune tecnologie e protocolli previsti dal modello risultavano acerbi, meno efficienti o poco pratici rispetto alle soluzioni già operative.
- **Implementazioni complesse o inefficienti:** molte implementazioni si sono rivelate difficili da integrare e poco competitive in termini di prestazioni. Ad esempio, nel modello ISO/OSI:
 - Al livello *Trasporto*, le connessioni possono essere sia orientate che non orientate alla connessione.
 - Al livello *Datalink*, invece, le connessioni sono sempre orientate alla connessione.
- **Fattori politici e burocratici:** l'eccessiva frammentazione tra enti normativi, la standardizzazione lenta e i conflitti tra interessi commerciali hanno ostacolato l'adozione diffusa del modello.

1.6.2 Modello TCP/IP

Il modello **TCP/IP**, che prende il nome dai suoi due protocolli principali, rappresenta l'architettura di riferimento utilizzata per Internet e nasce dallo sviluppo della rete ARPANET. Questo modello fu progettato con l'obiettivo di interconnettere reti eterogenee e garantire un'architettura flessibile e scalabile.

Il modello TCP/IP si compone di quattro livelli principali:

- **Applicazione:** fornisce i servizi di rete direttamente agli utenti e alle applicazioni (es. HTTP, FTP, DNS, SMTP).
- **Trasporto:** garantisce il trasferimento affidabile o non affidabile dei dati tra due host. I principali protocolli sono:
 - **TCP** (Transmission Control Protocol): orientato alla connessione, assicura l'affidabilità e l'ordine dei dati.
 - **UDP** (User Datagram Protocol): non orientato alla connessione, più veloce ma senza garanzie di consegna o ordine.
- **Internet:** gestisce l'instradamento e l'indirizzamento dei pacchetti all'interno della rete, permettendo che raggiungano la destinazione anche attraversando reti diverse. Il protocollo principale è:
 - **IP** (Internet Protocol), che può essere affiancato da protocolli ausiliari come ICMP (Internet Control Message Protocol) per la gestione di errori e segnalazioni.
- **Link:** definisce le modalità di accesso al mezzo trasmissivo e si occupa della trasmissione fisica dei dati tra nodi adiacenti.

1.6.2.1 Criticità

Nonostante il modello TCP/IP sia diventato lo standard de facto per le reti, presenta alcune limitazioni strutturali:

- **Assenza di distinzione chiara tra interfaccia, servizi e protocollo:** il modello non separa in modo esplicito questi concetti, generando ambiguità nelle implementazioni e nelle specifiche.
- **Poco generalizzato:** il modello è stato progettato in modo specifico per Internet e si adatta meno facilmente a contesti o tecnologie differenti.
- **Ambiguità nel livello di *link*:** il livello di collegamento viene trattato più come un'interfaccia tra il livello *Internet* e il mezzo trasmissivo, piuttosto che come un layer completo con responsabilità definite.
- **Mancanza di distinzione tra livello fisico e livello di collegamento:** nel modello TCP/IP questi due livelli, separati nel modello ISO/OSI, vengono spesso fusi o trattati con poca chiarezza.

1.6.3 Modello Ibrido

Per superare le criticità dei modelli ISO/OSI e TCP/IP, si è diffuso un modello **ibrido**, che integra i punti di forza di entrambi. Questo modello è composto da 5 livelli ben definiti:

- **Applicazione:** contiene i programmi e le applicazioni che utilizzano i servizi di rete (es. browser, client email, ecc.).
- **Trasporto:** fornisce servizi di consegna dati affidabili o non affidabili al livello superiore. Garantisce, se richiesto, il trasferimento ordinato e affidabile dei byte, nascondendo le complessità del livello di rete.
- **Network** (o Rete): si occupa dell'instradamento dei pacchetti tra sorgente e destinazione, combinando più link e dispositivi per permettere la comunicazione anche su lunghe distanze.
- **Datalink:** gestisce la comunicazione tra dispositivi direttamente connessi. Si occupa della trasmissione di frame, rilevamento e, se possibile, correzione degli errori locali.
- **Fisico:** descrive le caratteristiche fisiche e le modalità di trasmissione dei bit sul mezzo.

1.7 Esempi di reti

Alcuni esempi significativi di reti che hanno avuto un impatto storico o tecnologico rilevante sono i seguenti:

- **Internet:** è la più grande rete esistente, ma non è una rete unica e omogenea. Piuttosto, è un insieme interconnesso di milioni di reti eterogenee, pubbliche e private, che condividono protocolli standard. Internet fornisce servizi diffusi come il web, la posta elettronica, il trasferimento di file e le comunicazioni in tempo reale.
- **ARPANET:** nata negli Stati Uniti alla fine degli anni '60 su iniziativa dell'agenzia ARPA (Advanced Research Projects Agency), è considerata il precursore di Internet. ARPANET fu progettata per garantire la continuità delle comunicazioni anche in caso di guasti o attacchi a parte della rete, introducendo il concetto rivoluzionario di **commutazione a pacchetto**, che sostituì le reti basate sulla commutazione di circuito. La crescente diffusione della rete rese necessaria, negli anni '80, l'introduzione del **DNS** (Domain Name System), che consente di identificare i computer tramite nomi simbolici, semplificando l'accesso alle risorse di rete.
- **NSFNET** (National Science Foundation Network): sviluppata negli anni '80 come evoluzione e successore di ARPANET, con l'obiettivo di collegare università, istituti di ricerca e centri scientifici negli Stati Uniti. NSFNET ha avuto un ruolo chiave nella transizione verso un'infrastruttura di rete più estesa, scalabile e stabile, contribuendo in modo determinante alla nascita e alla successiva espansione globale di Internet.

1.7.1 Architettura di Internet

Per accedere a Internet è necessario stipulare contratti con appositi fornitori di connettività, chiamati **ISP** (Internet Service Provider). Gli ISP offrono diverse modalità di accesso, tra cui:

- **DSL** (Digital Subscriber Line): utilizza la linea telefonica tradizionale per trasmettere dati digitali. Sul lato dell'utente è necessario un modem DSL, che converte i pacchetti dati in segnali compatibili con la rete telefonica. All'altro capo, un dispositivo chiamato **DSLAM** (Digital Subscriber Line Access Multiplexer) riceve il segnale e lo riconverte in pacchetti per instradarli sulla rete.
- **Linea via cavo:** particolarmente diffusa negli Stati Uniti, sfrutta la rete delle trasmissioni televisive via cavo per fornire accesso a Internet. In questo caso si utilizza un **cable modem** lato utente, mentre lato ISP è presente il dispositivo **CMTS** (Cable Modem Termination System).
- **Connessioni wireless:** anche l'accesso senza fili a Internet avviene tramite gli ISP, che forniscono connettività tramite reti mobili, satellitari o altre tecnologie wireless.

Il punto fisico in cui i pacchetti dati accedono alla rete dell'ISP prende il nome di **POP** (Point of Presence). Gli ISP possono operare su scala locale, regionale, nazionale o internazionale. Le loro infrastrutture principali, composte da dorsali ad alta capacità, sono dette **backbone**.

Gli ISP sono interconnessi tra loro tramite strutture chiamate **IXP** (Internet Exchange Point), all'interno delle quali avviene il **peering**, ossia lo scambio diretto di traffico tra diverse reti.

Al vertice della gerarchia si trovano pochi grandi operatori globali, come **AT&T** e **Sprint**, che possiedono vaste dorsali internazionali basate su reti in fibra ottica a banda larga, con migliaia di router distribuiti. Questi operatori sono detti **Tier 1** e costituiscono l'ossatura fondamentale di Internet. I Tier 1 non pagano altri operatori per il transito dei dati, poiché tutti gli altri soggetti devono connettersi a loro per raggiungere l'intera rete globale.

Grandi aziende fornitrici di contenuti, come **Google** o **Yahoo!**, concentrano i propri server in apposite strutture dette **data center**, progettate per ospitare migliaia di dispositivi senza la presenza umana costante. Questi server sono organizzati in armadi chiamati **server farm**.

Inoltre, esistono data center dedicati al **colocation** o **hosting**, che permettono ai clienti di installare i propri server in prossimità dei POP degli ISP, riducendo la latenza e migliorando la velocità delle connessioni verso le dorsali principali.

2 Datalink Layer

Il **datalink layer** è responsabile della trasmissione affidabile dei dati tra due dispositivi direttamente collegati tramite un mezzo fisico di comunicazione.

Il suo compito principale è trasformare un canale fisico, che può essere soggetto a errori e interferenze, in un canale privo di errori apparenti per i livelli superiori. Per fare ciò, i dati vengono organizzati in unità chiamate **frame**.

2.1 Ethernet (IEEE 802.3)

Uno dei protocolli di livello datalink più diffusi è l'IEEE 802.3, noto anche come **Ethernet**. Questo standard prevede una divisione del livello datalink in due sottolivelli:

- **MAC (Media Access Control)**: gestisce l'accesso al mezzo trasmissivo condiviso e l'indirizzamento fisico tramite indirizzi MAC unici per ogni dispositivo.
- **LLC (Logical Link Control)**: fornisce servizi di controllo della connessione e gestione degli errori, offrendo un'interfaccia logica verso il livello di rete.

2.1.1 Frame

Il frame Ethernet presenta la seguente struttura:

- | | |
|---|-------------------|
| • Preamble | 7 byte |
| Serve a sincronizzare il ricevitore con il trasmettitore tramite una sequenza di bit alternati. | |
| • SFD (Start Frame Delimiter) | 1 byte |
| Indica l'inizio effettivo del frame, segnalando che i dati validi stanno per arrivare. | |
| • Destination address | 6 byte |
| Indirizzo MAC del dispositivo destinatario. | |
| • Source address | 6 byte |
| Indirizzo MAC del dispositivo mittente. | |
| • Length/Type | 2 byte |
| Indica la lunghezza del campo dati o il tipo di protocollo incapsulato. | |
| • Data e padding | da 46 a 1500 byte |
| Contiene i dati trasportati; se troppo corto, viene riempito con padding. | |
| • CRC (Cyclic Redundancy Check) | 4 byte |
| Codice di controllo per rilevare errori nei dati trasmessi. | |

2.1.2 Indirizzi

Gli indirizzi MAC sono identificatori univoci assegnati alle interfacce di rete. Hanno una lunghezza di 48 bit, ovvero 6 byte, rappresentati comunemente come 6 gruppi di due cifre esadecimali separati da due punti:

$$d_1d_2 : d_3d_4 : d_5d_6 : d_7d_8 : d_9d_{10} : d_{11}d_{12}$$

Gli indirizzi MAC possono essere classificati in tre categorie:

- **Unicast**: se il secondo nibble (d_2) del primo byte è pari (bit meno significativo a 0). Identifica un singolo dispositivo specifico sulla rete.
- **Multicast**: se il secondo nibble (d_2) del primo byte è dispari (bit meno significativo a 1). Rappresenta un gruppo di dispositivi che ricevono il frame.
- **Broadcast**: indirizzo speciale costituito da tutti bit a 1, ovvero **FF:FF:FF:FF:FF:FF**. Indica che il frame è destinato a tutti i dispositivi della rete locale.

Durante la trasmissione, gli indirizzi MAC vengono inviati in ordine **byte per byte** da sinistra verso destra. Tuttavia, all'interno di ogni singolo byte, i bit vengono trasmessi in ordine inverso (da destra verso sinistra).

2.1.3 Collisioni

Nel contesto delle reti Ethernet tradizionali con mezzo condiviso, le **collisioni** si verificano quando due dispositivi tentano di trasmettere simultaneamente sullo stesso canale. Per rilevare e gestire queste collisioni, Ethernet adotta il protocollo **CSMA/CD** (Carrier Sense Multiple Access with Collision Detection). Affinché il meccanismo di rilevamento delle collisioni funzioni correttamente, è essenziale che i frame abbiano una **dimensione minima** sufficiente. Questo garantisce che l'intero frame sia ancora in trasmissione quando un'eventuale collisione, generatasi nei punti più distanti della rete, può essere rilevata dal mittente.

2.1.3.1 CSMA/CD

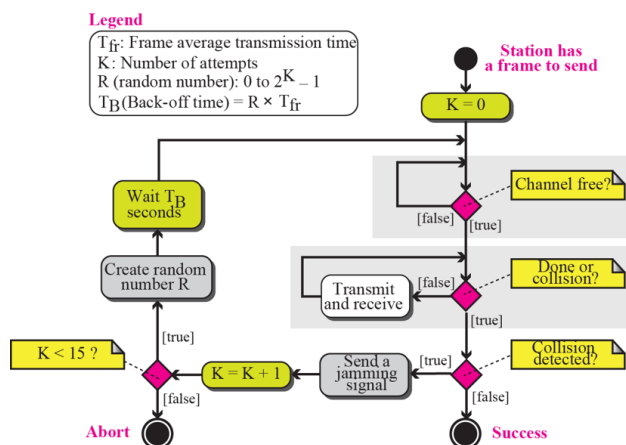
L'Ethernet classica utilizza il protocollo **CSMA/CD** (*Carrier Sense Multiple Access with Collision Detection*) con politica **1-persistente**.

- **Carrier Sense**: ogni stazione ascolta il canale prima di trasmettere; se il canale è occupato, attende che si liberi.
- **Multiple Access**: più stazioni condividono lo stesso mezzo trasmissivo.
- **Collision Detection**: mentre trasmette, una stazione continua a monitorare il canale per rilevare eventuali collisioni.

Nel dettaglio, il comportamento è il seguente:

1. Quando una stazione ha un frame da inviare, ascolta il canale.
2. Se il canale è libero, inizia immediatamente la trasmissione (comportamento 1-persistente).
3. Durante la trasmissione, la stazione continua a monitorare il canale.
4. Se rileva una collisione:
 - Interrompe immediatamente la trasmissione.
 - Invia un breve segnale caotico chiamato **jam signal** per garantire che tutte le altre stazioni riconoscano la collisione.
 - Attende un intervallo di tempo casuale, calcolato tramite l'algoritmo di **backoff esponenziale**, prima di ritentare la trasmissione.

Questo meccanismo riduce la probabilità di collisioni multiple consecutive e ottimizza l'uso del canale.



2.1.3.2 Dimensione minima frame

Il tempo necessario per trasmettere un frame deve essere almeno pari al **tempo di andata e ritorno** del segnale lungo la rete (round-trip time), che rappresenta il tempo massimo necessario perché una collisione possa propagarsi fino al mittente.

Se i frame fossero troppo brevi, un dispositivo potrebbe completare la trasmissione prima che un'eventuale collisione venga rilevata, compromettendo il corretto funzionamento del protocollo CSMA/CD.

2.1.4 Evoluzione

Il protocollo Ethernet ha subito una significativa evoluzione nella sua struttura fisica e logica, passando attraverso diverse topologie di rete:

- **Bus:** la prima versione di Ethernet utilizzava una topologia a bus condiviso, in cui tutti i nodi erano collegati allo stesso cavo coassiale. In questa configurazione, il canale era condiviso e il CSMA/CD era fondamentale per gestire le collisioni. I dispositivi venivano collegati al cavo tramite connettori a morsetto, comunemente chiamati “vampiri” perché si agganciavano direttamente al cavo principale senza interrompere la trasmissione.
- **Physical star:** successivamente, la rete è stata organizzata fisicamente a stella, con un hub centrale che collegava tutti i dispositivi. Sebbene fisicamente a stella, il mezzo trasmissivo era ancora condiviso, quindi il CSMA/CD restava necessario.
- **Logical star:** con l'introduzione degli switch, la rete mantiene una topologia fisica a stella, ma a livello logico ogni collegamento è dedicato punto-punto. Questo elimina la condivisione del mezzo e quindi la necessità di CSMA/CD.
- **Struttura gerarchica:** nelle reti più complesse, le topologie a stella vengono collegate tra loro in modo gerarchico, consentendo scalabilità e gestione efficiente del traffico.

2.1.5 Ethernet classica (10 Mbit/s)

La prima versione di Ethernet, con velocità massima di 10 Mbit/s, prevedeva principalmente due varianti di cablaggio:

- **Thick Ethernet (10Base5):** utilizza cavi coassiali spessi e robusti, può coprire distanze fino a 500 metri per segmento. È più costoso e meno flessibile, ma offre maggiore resistenza ai disturbi.
- **Thin Ethernet (10Base2):** utilizza cavi coassiali più sottili e connettori standard BNC. È più economico, flessibile e semplice da installare rispetto al Thick Ethernet, ma consente distanze massime inferiori, pari a 185 metri per segmento.

Indipendentemente dal tipo di cablaggio, è possibile estendere le reti tramite l'uso di **ripetitori**, dispositivi che amplificano e rigenerano il segnale tra due segmenti di rete. I ripetitori operano a livello fisico, utilizzano la codifica Manchester e permettono di superare le limitazioni di distanza dei singoli cavi.

Tuttavia, il protocollo Ethernet classico impone dei limiti strutturali:

- La distanza massima tra due dispositivi della rete non può superare i 2,5 km.
- Non possono essere presenti più di 4 ripetitori tra due stazioni qualsiasi della rete.

2.1.6 Fast Ethernet (100 Mbit/s)

L'obiettivo principale del **Fast Ethernet** era aumentare la velocità di trasmissione da 10 Mbit/s (Ethernet classica) a 100 Mbit/s, mantenendo il più possibile l'infrastruttura e l'architettura esistenti.

L'incremento delle prestazioni è stato ottenuto riducendo il **tempo di simbolo** da 100 ns a 10 ns. Tuttavia, tale aumento della velocità ha comportato un'espansione della **banda del segnale** di un fattore 10, rendendo inadeguati i cavi di categoria 3 utilizzati in precedenza.

Per questo motivo, il Fast Ethernet ha introdotto nuovi requisiti sul cablaggio:

- **Ethernet 100Base-TX:** utilizza cavi in rame di **categoria 5** (o superiore) con doppini intrecciati, consentendo trasmissioni affidabili fino a 100 Mbit/s.

Uno degli elementi chiave è l'utilizzo della **codifica 4B/5B**, che aumenta l'affidabilità della trasmissione rispetto alla semplice codifica binaria o alla codifica Manchester:

- La codifica 4B/5B converte ogni **gruppo di 4 bit** di dati in **5 bit** di segnale, introducendo ridondanza utile per la sincronizzazione e il rilevamento degli errori.

- Il segnale viene trasmesso a una frequenza di **125 MHz**, ottenendo:

$$\frac{4 \text{ bit dati}}{5 \text{ bit segnale}} \times 125 \text{ MHz} = 100 \text{ Mbit/s}$$

Lo standard 100Base-TX supporta sia la modalità **half-duplex** che **full-duplex**, consentendo il collegamento sia con hub che con switch. Tuttavia, per rispettare i vincoli imposti dal protocollo **CSMA/CD** in modalità half-duplex, si è dovuto ricorrere ad alcune modifiche strutturali:

- Aumentare la **lunghezza minima** del frame, in modo che il tempo di trasmissione sia superiore al tempo necessario per la propagazione del segnale e l'eventuale rilevazione di collisioni.
- Ridurre di un fattore 10 la **distanza massima** tra due dispositivi, rispetto all'Ethernet classica a 10 Mbit/s.

Nella pratica, fu adottata principalmente la seconda soluzione, ovvero la riduzione della distanza massima consentita tra due host.

Fast Ethernet include inoltre un meccanismo di **autonegotiation**, che consente ai dispositivi di negoziare automaticamente:

- La velocità di trasmissione (10 o 100 Mbit/s)
- La modalità operativa (half-duplex o full-duplex)

2.1.7 Gigabit Ethernet (1000 Mbit/s)

Come per il passaggio a Fast Ethernet, l'obiettivo principale di **Gigabit Ethernet** era aumentare le prestazioni di un fattore 10, mantenendo la massima compatibilità con le versioni precedenti.

Gigabit Ethernet supporta sia la modalità **full-duplex** che **half-duplex**:

- In modalità **full-duplex**, non vi è alcun limite specifico alla lunghezza del cavo imposto dal protocollo CSMA/CD, ma solo dai vincoli fisici legati all'attenuazione e alla potenza del segnale.
- In modalità **half-duplex**, per rispettare i vincoli temporali del CSMA/CD, si sarebbe dovuto ridurre la lunghezza massima del cavo a circa **25 metri** (un fattore 100 in meno rispetto a Ethernet classica). Tuttavia, sono state introdotte due tecniche per mitigare il problema:
 - **Carrier Extension**: estende artificialmente la lunghezza minima del frame fino a **512 byte**, garantendo che la trasmissione duri abbastanza a lungo per consentire il rilevamento di eventuali collisioni.
 - **Frame Bursting**: permette di inviare più frame concatenati in sequenza, migliorando l'efficienza complessiva della rete.

Per raggiungere velocità di trasmissione di **1 Gbps**, è necessario inviare un bit ogni nanosecondo. Le prime soluzioni adottate furono:

- **1000Base-CX**: utilizza cavi in rame schermati, adatti solo a brevi distanze.
- **1000Base-SX**: trasmissione su fibra ottica con lunghezza d'onda di **850 nm**, per brevi/medie distanze.
- **1000Base-LX**: trasmissione su fibra ottica con lunghezza d'onda di **1300 nm**, per lunghe distanze.

Per utilizzare **cavi in rame di Categoria 5** (o superiore), è stato necessario adottare un sistema di segnalazione più sofisticato, noto come **1000Base-T**. Le principali caratteristiche sono:

- Tutti e **quattro i doppi** del cavo vengono usati contemporaneamente, con trasmissione bidirezionale su ciascun doppino.
- Viene utilizzata una modulazione con **cinque livelli di tensione** (*PAM-5*), che consente di trasmettere più bit per simbolo.
- La velocità di simbolo è di **125 milioni di simboli al secondo**, con un sistema complesso di mappatura e codifica per garantire l'integrità del segnale, includendo tecniche di **correzione d'errore**.

2.1.8 10-Gigabit Ethernet (10GbE)

Le connessioni Ethernet a 10 Gigabit al secondo rappresentano un'evoluzione significativa rispetto alle versioni precedenti, progettate per garantire elevate prestazioni sia in ambito locale che su lunghe distanze.

- Per i collegamenti a lunga distanza si utilizza esclusivamente **fibra ottica**, grazie alla sua bassa attenuazione e capacità di trasmettere dati su chilometri senza rigeneratori.
- Per le distanze più brevi è possibile utilizzare sia **rame** che **fibra ottica**, anche se il rame presenta limiti maggiori in termini di distanza e qualità del segnale.

Tutte le versioni di 10-Gigabit Ethernet supportano esclusivamente la modalità **full-duplex**, eliminando la necessità di tecniche di controllo delle collisioni come il CSMA/CD, ormai superflue a queste velocità.

La compatibilità retroattiva rimane un requisito fondamentale: le interfacce di rete 10GbE implementano meccanismi di **auto-negotiation**, ovvero negoziazione automatica dei parametri di connessione. Se necessario, la comunicazione ripiega verso la velocità più elevata supportata da entrambe le estremità della connessione.

Dal punto di vista della trasmissione, tutte le versioni di 10GbE inviano un flusso sequenziale di dati, applicando una **codifica 64B/66B**, che introduce solo un leggero overhead (circa il 3%, molto inferiore rispetto alla codifica 8B/10B utilizzata nelle versioni precedenti). Questo consente una trasmissione più efficiente e affidabile a velocità così elevate.

2.1.9 Dispositivi di commutazione

I dispositivi di interconnessione si classificano in base al livello del modello OSI in cui operano:

- **Livello fisico** (*hub*): è un ripetitore multiporta che si limita a inoltrare i segnali ricevuti su tutte le porte, senza analizzare o interpretare il contenuto dei dati. Gli hub replicano i bit trasmessi a tutti i dispositivi connessi, creando un unico dominio di collisione condiviso. Non effettuano filtraggio né instradamento, il che può generare inefficienze nelle reti di grandi dimensioni.
- **Livello datalink** (*switch* o *bridge*): gli switch rappresentano un'evoluzione dei bridge e operano al livello 2 del modello OSI. Analizzano gli indirizzi MAC presenti nei frame per decidere su quale porta inoltrare i dati, migliorando l'efficienza rispetto agli hub. Ogni porta di uno switch rappresenta un dominio di collisione separato, riducendo il rischio di interferenze. Gli switch mantengono una tabella di indirizzi MAC (*switching table*) per associare ogni dispositivo alla porta corrispondente. Nelle comunicazioni full-duplex il protocollo CSMA/CD non è più necessario, mentre rimane in uso in modalità half-duplex.
- **Livello network** (*router*): i router operano al livello 3 e si occupano di instradare i pacchetti tra reti diverse sulla base degli indirizzi IP. Sono dispositivi fondamentali per connettere reti locali a reti esterne, come Internet, determinando il percorso ottimale per ciascun pacchetto verso la sua destinazione finale.

2.2 Wi-Fi (IEEE 802.11)

Il principale standard per le reti locali wireless (WLAN) è definito dall'IEEE 802.11, comunemente conosciuto come **Wi-Fi**.

Tutti gli standard della famiglia IEEE 802, incluso Ethernet, condividono una struttura simile per il livello **datalink**, che viene suddiviso in due sottolivelli:

- **Sottolivello MAC (Media Access Control)**: gestisce l'allocazione del canale radio e la trasmissione dei frame.
- **Sottolivello LLC (Logical Link Control)**: fornisce un'interfaccia uniforme verso il livello di rete, mascherando le differenze tra le diverse tecnologie della famiglia 802.

2.2.1 Modalità operative

Il protocollo IEEE 802.11 può funzionare principalmente in due modalità:

- **Modalità infrastrutturata (BSS - Basic Service Set):** ogni dispositivo wireless (stazione) si connette a un **Access Point** (AP), che funge da punto di accesso alla rete cablata. Più Access Point possono essere interconnessi attraverso un'infrastruttura cablata, denominata **sistema di distribuzione** (DS - Distribution System).
- **Modalità ad hoc:** i dispositivi wireless si connettono direttamente tra loro senza la necessità di un Access Point. Questa configurazione crea una rete temporanea tra i partecipanti, nota come **IBSS** (Independent Basic Service Set).

2.2.1.1 Extended Service Set (ESS)

Quando si uniscono più **BSS** (Basic Service Set) tramite un **sistema di distribuzione** (DS) e tutti gli **Access Point** (AP) condividono lo stesso identificatore di rete (SSID), si crea un **Extended Service Set** (ESS).

L'ESS consente di estendere la copertura della rete Wi-Fi su un'area più ampia, permettendo agli utenti di spostarsi tra diversi AP senza perdere la connessione, grazie al **roaming** trasparente.

2.2.2 Frame

Lo standard 802.11 definisce tre classi principali di frame:

- **Frame di dati:** trasportano il payload utile tra i dispositivi.
- **Frame di controllo:** coordinano l'accesso al mezzo e la gestione delle trasmissioni (ad esempio RTS/CTS, ACK).
- **Frame di gestione:** permettono la configurazione e il mantenimento delle connessioni (ad esempio Beacon, Association Request).

2.2.2.1 Dati (type = 10)

I **frame di dati** ha la seguente struttura generale:

- **Frame Control** 2 byte
Contiene informazioni di controllo generali per interpretare il frame.
 - **Version** 2 bit
Versione del protocollo 802.11.
 - **Type** 2 bit
Indica la tipologia del frame: dati, controllo o gestione.
 - **Subtype** 4 bit
Specifica la sotto-categoria del frame (ad esempio, Beacon, ACK, ecc.).
 - **To DS** 1 bit
Se attivo, indica che il frame è destinato al Distribution System (solitamente un AP).
 - **From DS** 1 bit
Se attivo, indica che il frame proviene dal Distribution System.
 - **More Fragments** 1 bit
Se attivo, indica che il frame è frammentato e ci sono altri frammenti da ricevere.
 - **Retry** 1 bit
Indica che il frame è una ritrasmissione di un frame precedente.
 - **Power Management** 1 bit
Indica lo stato di gestione energetica del dispositivo (ad esempio, modalità sleep).
 - **More Data** 1 bit
Il mittente ha altri dati in coda per il destinatario.

- **Protected Frame** 1 bit
Indica che il frame è protetto o cifrato (ad esempio, WPA/WPA2).
- **Order** 1 bit
Se attivo, richiede che il frame sia consegnato in ordine strettamente sequenziale.
- **Duration/ID** 2 byte
Specifica la durata stimata della trasmissione, compreso l'acknowledgment.
- **Destination address** 6 byte
Indirizzo MAC del destinatario del frame.
- **Source address** 6 byte
Indirizzo MAC del mittente del frame.
- **Address 3** 6 byte
Campo variabile a seconda dello scenario: può rappresentare l'indirizzo finale nel caso di rete a infrastruttura.
- **Sequence Control** 2 byte
Contiene il numero di sequenza e il numero di frammento, per la gestione della frammentazione e il riordinamento dei frame.
- **Address 4** 6 byte (opzionale)
Presente solo in particolari configurazioni.
- **Dati** 0 – 2312 byte
Contiene il payload utile, ovvero i dati trasportati dal frame. I primi byte sono sempre in formato LCC.
- **Frame Check Sequence (FCS)** 4 byte
Campo di controllo degli errori basato su CRC, per verificare l'integrità del frame.

Il significato e il numero degli indirizzi dipendono dal contesto e dal percorso dei dati (ad esempio, se passano attraverso un access point o tra due dispositivi in modalità ad hoc).

2.2.2.2 Gestione (type = 00)

I frame di gestione hanno lo stesso formato generale dei frame di dati, ma il contenuto della porzione **dati** varia in base al campo **Subtype**.

2.2.2.3 Controllo (type = 01)

I frame di controllo sono più brevi rispetto agli altri tipi di frame e si suddividono principalmente in:

- **RTS (Request to Send)**
Serve per richiedere l'accesso al canale trasmettendo una richiesta preliminare. La struttura è:
 - **Frame Control (FC)** 2 byte
Specifica il tipo e sottotipo del frame, oltre a varie flag di controllo.
 - **Duration/ID (D)** 2 byte
Indica il tempo previsto di utilizzo del canale, per riservarlo agli altri dispositivi.
 - **Address 1** 6 byte
Indirizzo di destinazione della richiesta (solitamente l'Access Point o il ricevente).
 - **Address 2** 6 byte
Indirizzo sorgente, ovvero chi sta richiedendo l'invio.
 - **Frame Check Sequence (FCS)** 4 byte
Campo di controllo per la rilevazione degli errori sul frame.
- **CTS (Clear to Send) / ACK (Acknowledgement)**
Rispondono rispettivamente a un RTS o confermano la ricezione di un frame correttamente. La struttura è:
 - **Frame Control (FC)** 2 byte
Specifica il tipo e sottotipo del frame, oltre a varie flag di controllo.

– Duration/ID (D)	2 byte
Riserva il canale per il tempo previsto.	
– Address 1	6 byte
Indirizzo del destinatario della risposta o della conferma.	
– Frame Check Sequence (FCS)	4 byte
Campo di controllo per la rilevazione degli errori.	

2.2.3 Condivisione

Quando un canale viene condiviso, lo standard **IEEE 802.11** definisce due approcci principali:

- **Polling (PCF - Point Coordination Function)**
In questo approccio, l'Access Point interroga ciclicamente i client chiedendo se hanno dati da trasmettere. Garantisce un accesso controllato e può essere utile per applicazioni che richiedono qualità del servizio (QoS).
- **Contention (DCF - Distributed Coordination Function)**
Le stazioni competono tra loro per accedere al canale utilizzando l'algoritmo **CSMA/CA** (Carrier Sense Multiple Access with Collision Avoidance), che cerca di evitare le collisioni anziché rilevarle come avviene nel CSMA/CD cablato.

Il **Polling (PCF)** e il **Contention (DCF)** possono coesistere simultaneamente nelle reti 802.11 ad infrastruttura. Tuttavia, nelle reti **ad hoc**, è disponibile esclusivamente il meccanismo **DCF**.

Nelle versioni recenti che supportano la **QoS** (come lo standard **IEEE 802.11e**), è stato introdotto il concetto di **Hybrid Coordination Function (HCF)**, che integra ed estende i meccanismi precedenti attraverso due modalità:

- **EDCA (Enhanced Distributed Channel Access)**
Evoluzione del **DCF**, introduce priorità differenziate per i frame, migliorando il supporto a traffico multimediale o sensibile al ritardo.
- **HCCA (HCF Controlled Channel Access)**
Evoluzione del **PCF**, fornisce un accesso controllato al canale con gestione avanzata delle risorse e delle priorità, particolarmente utile per applicazioni in tempo reale.

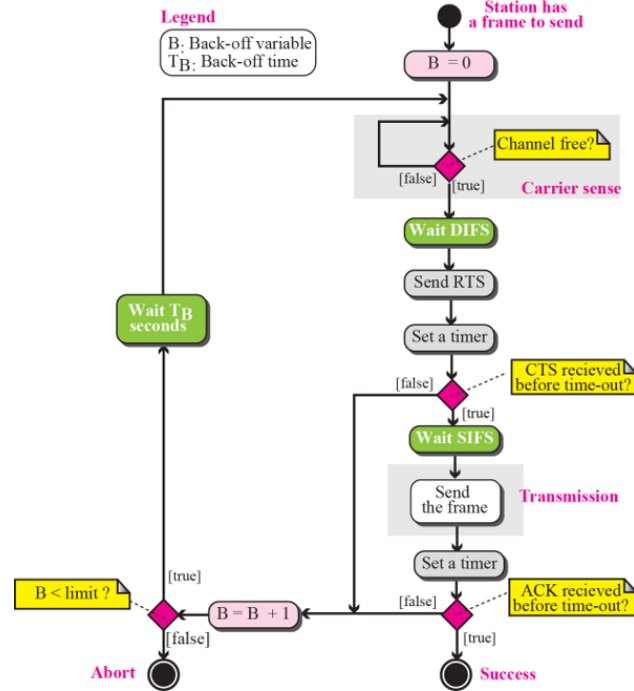
2.2.4 CSMA/CA

Il protocollo **CSMA/CA** (Carrier Sense Multiple Access with Collision Avoidance), a differenza del **CSMA/CD** utilizzato nelle reti cablate, non rileva le collisioni ma tenta di evitarle preventivamente, poiché nelle reti wireless non è sempre possibile rilevare una collisione durante la trasmissione.

Il funzionamento avviene secondo i seguenti passaggi:

- **Backoff Time**
Ogni stazione che deve trasmettere un frame genera un intervallo casuale di *backoff time*, espresso in unità di tempo chiamate *slot* ($slot \in [0, 15]$ in OFDM).
- **Ascolto del Canale**
La stazione rimane in ascolto e attende che il canale sia libero.
- **Countdown**
Quando il canale è rilevato come libero, il *backoff time* comincia a decrementare. Se durante il countdown un'altra trasmissione inizia, il timer si congela e riprende solo quando il canale torna libero.
- **Trasmissione**
Quando il *backoff time* raggiunge zero, la stazione trasmette il frame.

In un sistema di questo tipo, gli ACK vengono utilizzati per dedurre le collisioni.



2.2.5 Stazione nascosta

Il problema della **stazione nascosta** si verifica quando due stazioni non possono rilevare la trasmissione l'una dell'altra perché sono fuori portata, ma entrambe possono comunicare con un terzo nodo comune (ad esempio, un Access Point).

In questo scenario:

- Le due stazioni possono iniziare a trasmettere contemporaneamente al nodo comune, causando una collisione.
- Poiché non si vedono tra di loro, non possono evitare la collisione tramite il meccanismo CSMA/CA.

2.2.5.1 RTS/CTS

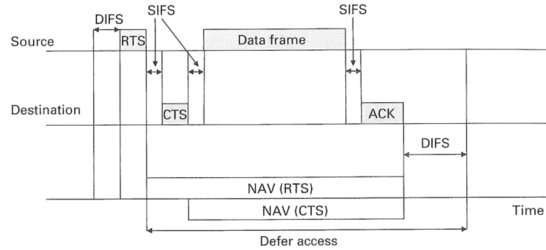
Per mitigare il problema della stazione nascosta, il protocollo 802.11 definisce anche una rilevazione fisica dell'utilizzo del canale. In particolare, ogni client possiede un **NAV** (*Network Allocation Vector*), il quale indica per quanto tempo (approssimativamente) il canale sarà occupato, se occupato.

Il valore di questo NAV viene impostato sulla base del campo **Duration** presente in ogni frame WiFi, che di conseguenza blocca automaticamente le trasmissioni degli altri client per tutta la durata specificata.

Il valore del NAV viene sfruttato dall'algoritmo **RTS/CTS** (*Request to Send / Clear to Send*) nel seguente modo:

- Una stazione invia un segnale **RTS** al destinatario della comunicazione.
- Quando il destinatario risponde con un **CTS**, il mittente può iniziare la trasmissione dei dati.
- Le stazioni che ricevono l'RTS ma non sono le destinatarie impostano il NAV, bloccando le proprie trasmissioni per il tempo specificato.
- Le stazioni che ricevono il CTS ma non sono le destinatarie impostano il NAV.
- Al termine della comunicazione, il destinatario invia un **ACK** al mittente. Questo ACK aggiorna il NAV delle altre stazioni, consentendo loro di riprendere le trasmissioni.

Tale tecnica non è tuttavia efficace con i frame corti (spediti al posto degli RTS) e con gli AP (che chiunque può raggiungere per definizione).



4.8 An RTS-CTS exchange and the corresponding setting of the network allocation vector

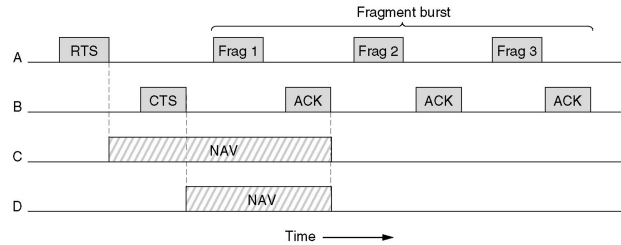
2.2.6 Frammentazione

Il protocollo 802.11 fornisce la qualità di servizio di divisione dei messaggi di grandi dimensioni in messaggi più piccoli, ognuno con il proprio checksum, detti **frammenti**.

Questi non hanno una dimensione prefissata ma possono essere gestiti (nelle dimensioni) dall'Access Point.

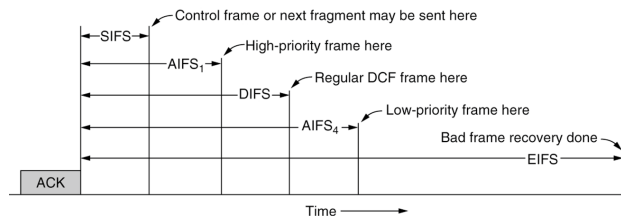
I frammenti sono numerati e soggetti a controllo attraverso l'algoritmo **Stop-and-Wait**.

Per ogni frammento, il destinatario invia un **ACK** al mittente, che potrà quindi spedire il frammento successivo.



Tuttavia, per evitare che l'ACK di un singolo frammento venga interpretato dagli altri dispositivi come un segnale di canale libero, si definiscono vari **offset temporali**, ovvero intervalli di tempo minimi che una stazione deve attendere prima di iniziare a trasmettere:

- **SIFS**
Short Interframe Space, il più breve, utilizzato tra frammenti consecutivi e tra frame e ACK.
- **AIFS₁** o **PIFS**
Arbitrary Interframe Space con priorità alta, usato per traffico prioritario. Può essere utilizzato, ad esempio, dagli Access Point.
- **DIFS**
Distributed Interframe Space, utilizzato da stazioni che vogliono iniziare una nuova trasmissione normale, in modalità DCF.
- **AIFS₄** o **PIFS**
Arbitrary Interframe Space con priorità bassa, usato per traffico meno prioritario o di background.
- **EIFS**
Extended Interframe Space, utilizzato in caso di errori di ricezione, prima di tentare una ritrasmissione.



Sulla base dei quattro livelli di priorità (ad eccezione di EIFS), vengono poi definiti **backoff time** differenti, per evitare collisioni e garantire maggiore equità nell'accesso al canale.

2.2.7 Stazione esposta

Il problema della **stazione esposta** si verifica quando una stazione deduce erroneamente che il canale sia occupato e, per questo motivo, rinuncia a trasmettere, nonostante non ci sarebbe interferenza reale.

Esempio: la stazione B vuole comunicare con C, ma rileva una trasmissione proveniente dalla stazione A. Tuttavia, A sta comunicando con un altro nodo che non è C, quindi B potrebbe comunque trasmettere senza interferenze, ma si astiene per errore.

2.2.7.1 Soluzioni di efficienza

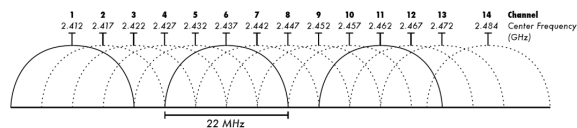
Per mitigare problemi come stazioni esposte e aumentare l'efficienza, lo standard 802.11 introduce:

- **OFDM (Orthogonal Frequency Division Multiplexing)**

I dati vengono trasmessi su 52 sottoportanti parallele: 48 per i dati e 4 per la sincronizzazione. Ogni simbolo dura 4 microsecondi e trasporta 1, 2, 4 o 6 bit.

I dati sono protetti con codifica per la correzione degli errori (con efficienze di $1/2$, $2/3$ o $3/4$).

Lo standard 802.11a, basato su OFDM, raggiunge velocità tra 6 e 54 Mbps nella banda a 5 GHz, con meno interferenze ma raggio ridotto rispetto a 802.11b.



- **802.11n**

Evoluzione approvata nel 2009, punta a un throughput superiore a 100 Mbps reali. Introduce:

- Canali raddoppiati da 20 MHz a 40 MHz
- Riduzione degli overhead
- Invio di frame aggregati
- **MIMO** (Multiple Input Multiple Output), che sfrutta più antenne per trasmettere più flussi simultanei, migliorando velocità, copertura e affidabilità

3 Network layer

Il network layer è incaricato dell'istadamento dei pacchetti.

3.1 Tipi di indirizzi

I tipi di indirizzi si differenziano tra i layer:

- **Data Link:** indirizzi fisici, come gli indirizzi MAC. Sono univoci per ogni scheda di rete.
- **Network:** indirizzi logici, come gli indirizzi IP. Sono dinamici e possono cambiare nel tempo. Gli indirizzi IP sono univoci all'interno di una rete e possono essere associati a un indirizzo MAC tramite la tabella ARP. È compito del protocollo ARP (posto tra il livello 2 e il livello 3) completare e aggiornare tale tabella.
- **Transport:** indirizzi applicativi, come le **porte**. Le porte identificano in modo univoco un processo o servizio all'interno di un host.

3.2 Switching

Lo switching consiste nel decidere quale porta utilizzare per instradare un pacchetto. Esistono due principali modalità:

- **Circuit switching:** viene stabilito un circuito fisico dedicato tra la sorgente e la destinazione per tutta la durata della comunicazione.
- **Packet switching:** il messaggio viene suddiviso in pacchetti ricostruiti una volta giunti a destinazione.

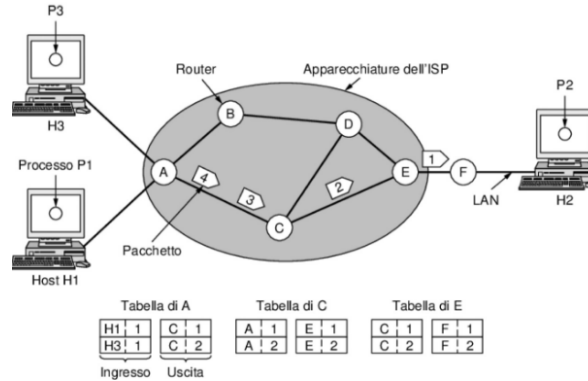
3.3 Connessione

L'istadamento dei pacchetti nei casi di **packet switching** può essere con o senza connessione:

- **Senza connessione:** ogni pacchetto, in questo caso detto **datagram**, viene instradato indipendentemente dagli altri. Il percorso verso una stessa destinazione può infatti variare a seconda dell'algoritmo di routing o dallo stato delle tabelle di routing degli hop intermedi. L'ordine di arrivo non è garantito e ogni viaggio è discriminato esclusivamente dalla destinazione.

Destinazione	Linea (prossimo hop)
B	B (Diretto)
C	C (Diretto)
D	B
E	C
F	B

- **Con connessione** (orientato alla connessione): prima di inviare i pacchetti, viene stabilita una connessione (un percorso) detto **virtual circuit**. Il VC viene generato allo stabilire della connessione ed è persistente in tutti gli hop attraversati fino alla chiusura della connessione. Ogni VC viene associato ad un identificativo, utilizzato da tutti i pacchetti della connessione. In caso di conflitto (più VC con lo stesso identificatore), è l'hop che può ancora distinguere la provenienza dalle due reti diverse a modificare il valore dell'identificatore di una delle due VC da quel punto in poi delle rete virtuale (**commutazione di etichetta**).



All'interno di Internet si usa esclusivamente il metodo **packet switching** di tipo **connectionless**.

3.3.1 Creazione Virtual Circuit

La creazione di un **Virtual Circuit** (VC) avviene prima della trasmissione dei dati ed è costituita da tre fasi principali:

1. Setup (Stabilimento del percorso):

La sorgente invia una richiesta di connessione che attraversa i nodi intermedi fino alla destinazione. Ogni nodo lungo il percorso, ricevuta la richiesta, assegna un **identificatore univoco di VC**, spesso chiamato **label**. Questa label è associata a una specifica **porta di uscita** (interfaccia di rete) attraverso cui il pacchetto deve essere instradato per raggiungere il prossimo hop.

Il nodo salva nella propria tabella di commutazione l'associazione tra la label in ingresso (quella ricevuta dal nodo precedente) e la label in uscita (quella da assegnare al prossimo hop), insieme alla porta di uscita corrispondente. Questo processo è noto come **commutazione di etichetta**.

2. Trasmissione dei dati:

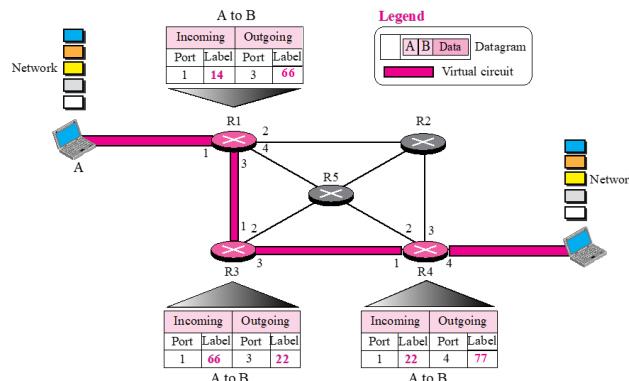
Una volta stabilito il VC, ogni pacchetto contiene solo la label assegnata (e non più l'indirizzo completo), che identifica il circuito virtuale lungo cui deve viaggiare. Ad ogni nodo, la label in ingresso viene utilizzata per consultare la tabella di commutazione, che sostituisce la label con quella corretta per l'uscita successiva e invia il pacchetto attraverso la porta di uscita associata.

Questo meccanismo riduce il carico computazionale di routing perché i nodi non devono più analizzare l'indirizzo di destinazione completo, ma si basano solo sulle label e sulla tabella di forwarding.

3. Teardown (Chiusura del circuito):

Al termine della comunicazione, viene inviato un messaggio di chiusura del VC in modo retrogrado lungo il percorso. Ogni nodo rimuove le associazioni di label e libera le risorse (porte, buffer) riservate per quel circuito.

In sintesi, la gestione delle **label** e delle **porte di uscita** è fondamentale per mantenere attivo il VC. Le label agiscono come identificatori temporanei e locali, mentre le porte indicano il percorso fisico da seguire in ogni nodo, garantendo che i pacchetti viaggino lungo un percorso stabilito e coerente.



3.4 Tabelle

3.4.1 ARP

Ogni scheda di rete possiede un indirizzo MAC univoco e un indirizzo logico (IP), quest'ultimo assegnato staticamente o tramite DHCP. L'associazione tra indirizzo IP e MAC viene gestita tramite la tabella ARP (Address Resolution Protocol), presente su ogni dispositivo connesso a una rete locale, per consentire la comunicazione all'interno della stessa.

Il funzionamento del protocollo ARP è il seguente:

- Un host che deve inviare un pacchetto a un indirizzo IP e non conosce il corrispondente indirizzo MAC invia una **ARP Request** in broadcast sulla rete locale. Questa contiene l'indirizzo IP di destinazione e richiede l'indirizzo MAC corrispondente.
- Tutti i dispositivi sulla LAN ricevono la richiesta, ma solo quello con l'indirizzo IP richiesto risponde con una **ARP Reply** in unicast, fornendo il proprio indirizzo MAC.
- L'host mittente aggiorna la propria tabella ARP con la nuova associazione IP-MAC, evitando future richieste per lo stesso indirizzo.

Le voci nella tabella ARP hanno durata temporanea e vengono eliminate dopo un periodo di inattività, per mantenere aggiornate le associazioni.

3.4.2 Routing

Ogni router possiede una tabella di routing che indica il prossimo hop da seguire per raggiungere le diverse destinazioni di rete. Questa tabella può essere configurata manualmente (**static routing**) oppure aggiornata dinamicamente tramite protocolli di routing come OSPF, BGP, e altri.

Nei protocolli di routing dinamico, ogni router scambia informazioni con i router adiacenti riguardo alle reti conosciute. Questi scambi permettono di aggiornare periodicamente la tabella di routing.

Le informazioni di routing possono essere propagate in due modalità principali:

- **Broadcast o Multicast:** aggiornamenti inviati a tutti i router adiacenti.
- **Unicast:** aggiornamenti inviati direttamente a uno specifico router.

Grazie alla tabella di routing, un router è in grado di determinare il percorso verso una destinazione anche se questa si trova su una rete remota, non direttamente connessa. Per ogni pacchetto, in base all'indirizzo di destinazione, il router è così in grado di identificare la scheda di rete locale attraverso cui inoltrare il pacchetto (se il destinatario appartiene alla rete locale) oppure l'indirizzo del prossimo hop (detto **gateway**) verso la destinazione finale.

3.4.3 MTU

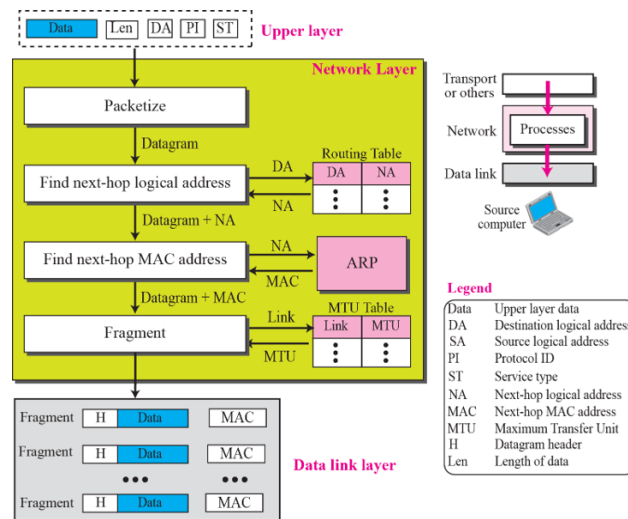
L'**MTU** (Maximum Transfer Unit) rappresenta la dimensione massima, in byte, di un pacchetto che può essere trasmesso su un determinato collegamento senza frammentazione.

Esiste una tabella che indica, per ogni percorso o interfaccia di rete, il valore MTU associato. Questo permette di frammentare il datagram, se necessario.

3.5 Protocolli di livello 3

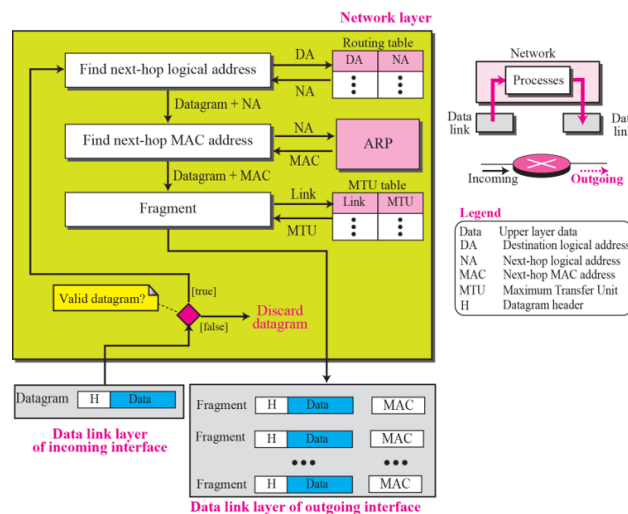
Il processo all'interno del livello 3 può essere riassunto come segue:

3.5.1 Invio



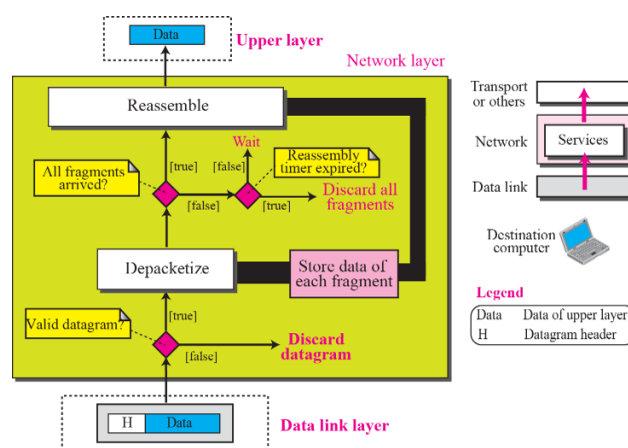
1. I dati provenienti dal livello di trasporto vengono inseriti in un pacchetto chiamato **datagram**.
2. Si determina l'indirizzo IP del prossimo hop consultando la **tabella di routing**.
3. Si determina l'indirizzo MAC del prossimo hop tramite la **tabella ARP**, utilizzando l'indirizzo IP ottenuto al passo precedente.
4. Si verifica la dimensione massima consentita tramite la **MTU table**; se il datagram supera tale dimensione, viene frammentato.

3.5.2 Hop



1. Il datagram viene ricevuto dal livello **Data Link** e ne viene verificata la validità. Se il pacchetto è corrotto o non conforme, viene scartato.
2. Si consulta la **tabella di routing** per determinare l'indirizzo IP del prossimo hop.
3. Si utilizza la **tabella ARP** per ottenere l'indirizzo MAC del prossimo hop.
4. Si verifica la dimensione massima consentita (MTU); se necessario, il datagram viene frammentato.

3.5.3 Ricezione



1. Il datagram viene ricevuto dal livello **Data Link** e ne viene verificata la validità. I pacchetti corrotti vengono scartati.
2. Il datagram viene depacchettizzato:
 - (a) I dati utili vengono estratti e memorizzati.
 - (b) Si attende l'arrivo di tutti i frammenti.
 - (c) Se il timer massimo per il riassettaggio scade, tutti i frammenti parziali vengono eliminati.
 - (d) Se tutti i pacchetti sono stati ricevuti, si procede
3. Una volta ricevuti tutti i frammenti, il pacchetto viene riassetato e i dati vengono passati al livello superiore.

3.6 Protocollo IP

3.6.1 Versione IPv4

L'indirizzo IPv4 è un indirizzo a 32 bit. Rappresenta una connessione di rete, non un singolo dispositivo (che può avere più indirizzi IP). Ha validità topologica, ovvero indica la posizione dell'interfaccia nella rete. L'indirizzo è diviso in due parti:

- **Prefisso** (o **network**): identifica univocamente una rete l bit
- **Suffisso** (o **host**): identifica univocamente un host all'interno della rete $32 - l$ bit

Per trovare l'indirizzo di rete a partire da un indirizzo IP si utilizza una **subnet mask**, che ha la forma di un indirizzo IP con i primi l bit impostati a 1 (corrispondenti al prefisso di rete) e gli altri a 0. Effettuando un AND bit a bit tra l'indirizzo IP e la subnet mask si ottiene l'indirizzo di rete.

Questa tecnica di progettazione è detta **classless addressing**.

3.6.1.1 Storia

Inizialmente il protocollo IP adottava il **classful addressing**, con cinque classi di indirizzi:

1. **Classe A**: indirizzi che iniziano con 0, con un campo rete di 8 bit e campo host di 24 bit
2. **Classe B**: indirizzi che iniziano con 10, con campo rete di 16 bit e host di 16 bit
3. **Classe C**: indirizzi che iniziano con 110, con campo rete di 24 bit e host di 8 bit
4. **Classe D**: indirizzi che iniziano con 1110, riservati al multicast
5. **Classe E**: indirizzi che iniziano con 1111, riservati a scopi futuri o sperimentali

La differenza principale tra le classi era la lunghezza fissa del campo rete e host

3.6.1.2 Utilizzo

Alcuni indirizzi IPv4 hanno usi speciali:

- **0.0.0.0**: indirizzo utilizzato dagli host all'avvio, quando l'indirizzo IP non è ancora assegnato.
- Indirizzi con bit di rete a zero: rappresentano la rete corrente (necessaria la conoscenza della subnet mask).
- **255.255.255.255**: indirizzo broadcast limitato, usato per inviare pacchetti a tutti gli host della rete locale.
- Indirizzo di rete normale con **bit di host tutti a uno**: broadcast verso tutti gli host della rete specificata, eseguibile anche dall'esterno.
- **127.0.0.0/8**: intervallo di indirizzi di loopback, utilizzati per test interni al dispositivo. Un pacchetto con indirizzo di loopback non raggiunge mai il livello 2.

3.6.1.3 Visibilità

Gli indirizzi IP privati, utilizzati solo all'interno di reti locali e non instradabili su Internet, appartengono ai seguenti intervalli:

- 10.0.0.0/8 (da 10.0.0.0 a 10.255.255.255)
- 172.16.0.0/12 (da 172.16.0.0 a 172.31.255.255)
- 192.168.0.0/16 (da 192.168.0.0 a 192.168.255.255)
- 169.254.0.0/16 (da 169.254.0.0 a 169.254.255.255)

3.6.1.4 Struttura

L'header di un datagramma IPv4 ha una lunghezza minima di 20 byte e massima di 60 byte, ed è composto dai seguenti campi:

- **Version** 4 bit
Indica la versione del protocollo IP (per IPv4 il valore è 4).
- **HLEN (Header Length)** 4 bit
Lunghezza dell'header espressa in multipli di 4 byte. Valore minimo 5 (ovvero 20 byte).
- **Service Type (DSCP + ECN)** 8 bit
Campo utilizzato per la qualità del servizio. I primi 6 bit sono il **DSCP** (Differentiated Services Code Point), che definisce la priorità o il trattamento preferenziale dei pacchetti; i 2 bit meno significativi sono l'**ECN** (Explicit Congestion Notification), utilizzato per segnalare la congestione della rete.
- **Total Length** 16 bit
Dimensione totale del datagramma IP (header + dati), espressa in byte.
- **Identification** 16 bit
Identificativo unico del datagramma, utile per la frammentazione e il riassemblaggio dei pacchetti.
- **Flags** 3 bit
Controlla la frammentazione:
 - Bit 0: riservato, deve essere 0
 - **DF** (Don't Fragment): se impostato, il pacchetto non può essere frammentato
 - **MF** (More Fragments): se impostato, il pacchetto ha altri frammenti successivi
- **Fragment Offset** 13 bit
Specifica la posizione del frammento rispetto all'inizio del datagramma originale, in unità da 8 byte.
- **Time To Live (TTL)** 8 bit
Numero massimo di hop consentiti per il pacchetto. Viene decrementato a ogni router per evitare loop infiniti.
- **Protocol** 8 bit
Specifica il protocollo del livello superiore incapsulato (ad esempio TCP=6, UDP=17).

- **Header Checksum** 16 bit
Valore di controllo per verificare l'integrità dell'header IP.
- **Source Address** 32 bit
Indirizzo IP sorgente.
- **Destination Address** 32 bit
Indirizzo IP di destinazione.
- **Options + Padding** fino a 40 byte
Campi opzionali per funzionalità avanzate. Se necessario, viene aggiunto padding per garantire che l'header sia un multiplo di 4 byte.
Le opzioni possono includere:
 - **Record Route**
Memorizza l'indirizzo IP di uscita di ogni hop attraversato. La struttura prevede:
 - * 1 byte **Type**
 - * 1 byte **Length**
 - * 1 byte **Pointer**, indica la prossima posizione libera per scrivere un nuovo indirizzo
 - **Strict Source Route**
Specifica un percorso rigido che il pacchetto deve seguire. Stessa struttura:
 - * 1 byte **Type**
 - * 1 byte **Length**
 - * 1 byte **Pointer**
 - **Loose Source Route**
Suggerisce un percorso, consentendo deviazioni parziali. Struttura analoga:
 - * 1 byte **Type**
 - * 1 byte **Length**
 - * 1 byte **Pointer**
 - **Timestamp**
Registra i timestamp degli hop attraversati, con la possibilità di includere anche gli indirizzi IP.
La struttura comprende:
 - * 1 byte **Type**
 - * 1 byte **Length**
 - * 1 byte **Pointer**
 - * 4 bit **Overflow**
 - * 4 bit **Flags** che determinano il formato (solo timestamp, timestamp + IP, ecc.)

3.6.2 Versione IPv6

Protocollo con indirizzi a 128 bit. Non può interagire direttamente con il protocollo IPv4.

3.6.2.1 Notazione

Gli indirizzi IPv6 vengono scritti come 8 gruppi di 4 cifre esadecimali separati da due punti (:). È possibile abbreviare sequenze di zeri con "::" (una sola volta per indirizzo).

3.6.2.2 Struttura

- **Version** 4 bit
Indica la versione del protocollo IP (per IPv6 è 6).
- **Traffic Class** 8 bit
I sei bit più significativi vengono usati per differenziare i tipi di servizi. I due meno significativi, invece, per l'ECN.

• Flow Label	20 bit
Identifica un flusso di pacchetti che devono essere trattati allo stesso modo, in maniera tale da evitare il riordinamento.	
• Payload Length	16 bit
Lunghezza del payload (dati) successivo all'header IPv6, in byte.	
• Next Header	8 bit
Indica il tipo di header successivo (es. TCP, UDP, estensioni).	
• Hop Limit	8 bit
Numero massimo di hop che il pacchetto può attraversare, decrementato a ogni router.	
• Source Address	128 bit
Indirizzo IPv6 sorgente.	
• Destination Address	128 bit
Indirizzo IPv6 destinazione.	

3.6.3 Differenze tra gli header IPv4 e IPv6

- **IHL**: rimosso in IPv6, poiché l'header ha lunghezza fissa (40 byte).
- **Protocol**: sostituito dal campo **Next Header** in IPv6, che indica il protocollo o header successivo.
- **Frammentazione**: rimossa da IPv6; la frammentazione è gestita solo dal mittente, non dai router. Gli hop intermedi, quindi, non possono più frammentare.
- **Checksum**: eliminato in IPv6 per ridurre il carico di elaborazione sui router.

3.7 NAT

Il protocollo NAT (Network Address Translation) permette di tradurre tutti gli indirizzi IP appartenenti a una sottorete interna in un unico indirizzo IP pubblico esterno. Gli indirizzi interni sono univoci solo all'interno della rete locale. Per consentire a tutti gli host interni di comunicare con l'esterno, viene quindi utilizzato un unico indirizzo IP pubblico, condiviso tramite NAT.

Proprio in questo contesto si evidenzia la distinzione fondamentale tra indirizzi IP pubblici e privati.

3.7.1 Processo decisionale

Il protocollo NAT sfrutta le porte per identificare le comunicazioni tra un host interno alla rete e un host esterno. Ad esempio, quando un processo si connette via TCP a un processo su un'altra macchina, utilizza una porta sorgente locale e una porta di destinazione remota.

Quando un pacchetto raggiunge il dispositivo NAT, questo sostituisce l'indirizzo IP interno con l'indirizzo IP pubblico e modifica la porta sorgente con un nuovo valore, che fa riferimento a una tabella interna di traduzioni. Tale tabella memorizza la corrispondenza tra coppia IP e porta originali (interne) e coppia IP e porta modificate (esterne), consentendo al NAT di instradare correttamente le risposte verso l'host interno. Quando un pacchetto di risposta viene ricevuto, il dispositivo NAT consulta la tabella di traduzioni utilizzando la porta di destinazione e ripristina l'indirizzo IP e la porta originali, in modo che il pacchetto venga consegnato correttamente all'host interno. Generalmente, il NAT conserva tabelle diverse per comunicazioni TCP e UDP.

Questa modalità, che coinvolge anche la traduzione delle porte, è detta **NAPT** (Network Address and Port Translation). La versione base del NAT si limita alla sostituzione dell'indirizzo IP, senza alterare le porte. In pratica, però, i due concetti vengono spesso identificati con il termine generico **NAT**.

3.7.2 PAT

La tecnica di traduzione delle porte viene specificamente introdotta dal protocollo **PAT** (Port Address Translation). Come descritto, PAT combinato con il NAT consente la traduzione simultanea di indirizzo IP e porta, realizzando così il **NAPT**.

3.7.3 Ulteriori utilizzi

I dispositivi NAT permettono anche di filtrare i pacchetti in ingresso, scartando quelli non richiesti. Di fatto, può anche assumere la funzione di firewall.

3.8 Multiplexing

Oltre al protocollo IP, il livello di rete supporta altri protocolli noti come protocolli di **layer 3+**, in quanto si basano direttamente su IP, ma forniscono funzionalità aggiuntive. Tra questi si includono, ad esempio, i protocolli **ICMP**, **IGMP**, **OSPF** e altri, che svolgono funzioni di controllo, gestione o instradamento della rete.

4 Transport layer

Il livello di trasporto è incaricato della consegna di pacchetti end-to-end.

4.1 Servizi

Il livello di trasporto offre due principali servizi:

- **UDP**: non orientato alla connessione. Spedisce semplicemente pacchetti tra le applicazioni, permettendo la costruzione di protocolli proprietari
- **TCP**: orientato alla connessione. Crea connessione e aumenta l'affidabilità attraverso le ritrasmissioni. Implementa i controlli di flusso e di congestione.

4.2 RPC

Il protocollo RPC (Remote Procedure Call) consente a un processo su una macchina di invocare una procedura che viene eseguita su un'altra macchina della rete, come se fosse una normale chiamata locale. Nel modello RPC, il processo che effettua la richiesta è detto **client**, mentre quello che implementa la procedura remota è detto **server**. Il meccanismo si basa sui seguenti componenti:

- **Client Stub**: modulo locale al client che incapsula la chiamata remota e la converte in una richiesta di rete.
- **Server Stub**: modulo locale al server che riceve la richiesta, la converte in una chiamata locale e restituisce il risultato al client.

Il flusso di una chiamata RPC è il seguente:

1. Il client esegue una normale chiamata alla procedura dichiarata nel **client stub**.
2. Il **client stub** prepara la richiesta, serializza i parametri (marshalling) e la invia attraverso la rete.
3. Il **server stub** riceve il messaggio, deserializza i parametri e invoca la procedura locale corrispondente.
4. Il risultato viene riconfezionato dal **server stub** e inviato al client.
5. Il **client stub** riceve la risposta e la restituisce come risultato della chiamata locale.

Il tutto avviene in trasparenza per il programma client, che invoca la procedura remota come se fosse una funzione locale, senza occuparsi dei dettagli legati alla comunicazione di rete.

4.3 UDP

Il **User Datagram Protocol** (UDP) fornisce un servizio di trasporto semplice, leggero e non affidabile, basato su un'unità minima detta **datagramma**. A differenza di TCP, UDP non garantisce l'ordinamento, l'affidabilità né il controllo di congestione, offrendo invece un trasferimento diretto dei dati con minima latenza e overhead ridotto.

Ogni datagramma UDP è composto da un **header** di 8 byte seguito da un **payload** di lunghezza variabile.

4.3.1 Servizi

- **Multiplexing** e **demultiplexing** basati su numeri di porta.
- Supporto per comunicazioni:
 - **Unicast** (uno a uno)
 - **Multicast** (uno a molti)
 - **Broadcast** (uno a tutti nella rete locale)
- **Rilevamento degli errori** tramite checksum a 16 bit opzionale.

4.3.2 Header

L'header UDP è composto da 4 campi, ciascuno di 16 bit:

- **Source Port** 16 bit
Porta sorgente, utilizzata per identificare il processo applicativo mittente sul nodo di origine.
- **Destination Port** 16 bit
Porta di destinazione, utilizzata per identificare il processo applicativo ricevente sul nodo di destinazione.
- **Length** 16 bit
Lunghezza totale del segmento UDP, espressa in byte, comprensiva sia dell'header (8 byte) sia dei dati trasportati.
- **Checksum** 16 bit
Campo utilizzato per il controllo d'integrità del segmento. Il valore è calcolato come complemento a due della somma dei seguenti elementi, considerando coppie di byte:
 - Header UDP
 - Dati (se presenti, con padding se necessario per completare multipli di 16 bit)
 - Pseudo-header IP, composto da:
 - * Indirizzo IP sorgente (IPv4 o IPv6)
 - * Indirizzo IP di destinazione (IPv4 o IPv6)
 - * Valore del protocollo (17 per UDP)
 - * Lunghezza totale del segmento UDP

Il checksum è obbligatorio in IPv6, mentre in IPv4 può essere opzionale: in tal caso il campo viene impostato a zero.

4.3.3 Utilizzi

Il protocollo UDP viene utilizzato in tutti i casi in cui la **velocità di trasferimento** e la **bassa latenza** sono prioritari rispetto alla garanzia di consegna dei dati. È indicato per applicazioni che possono tollerare la perdita di alcuni pacchetti, ma richiedono tempi di risposta rapidi.

Esempi tipici di utilizzo:

- **VoIP** (Voice over IP): trasmissione vocale in tempo reale.
- **Streaming multimediale** (ad esempio, RTP - Real-Time Protocol).
- **Protocolli proprietari** per comunicazioni in tempo reale o applicazioni specializzate.
- **DNS** (Domain Name System): risoluzione rapida di nomi di dominio.
- **DHCP** (Dynamic Host Configuration Protocol): assegnazione dinamica degli indirizzi IP.
- **SNMP** (Simple Network Management Protocol): monitoraggio e gestione remota di dispositivi di rete.
- **RIP** (Routing Information Protocol): protocollo di routing basato su UDP.

4.3.4 RTP

Il protocollo RTP (Real-time Transport Protocol) è progettato per la trasmissione di dati in tempo reale su reti IP, come audio, video o altri contenuti multimediali. Viene generalmente implementato nello spazio utente e si appoggia su UDP, sfruttandone la bassa latenza.

RTP fornisce servizi di trasporto utili per applicazioni real-time ed è, per esempio, sfruttato nel VoIP.

4.3.4.1 Header

L'header di un pacchetto RTP ha lunghezza variabile, con un minimo di 12 byte, e contiene i seguenti campi:

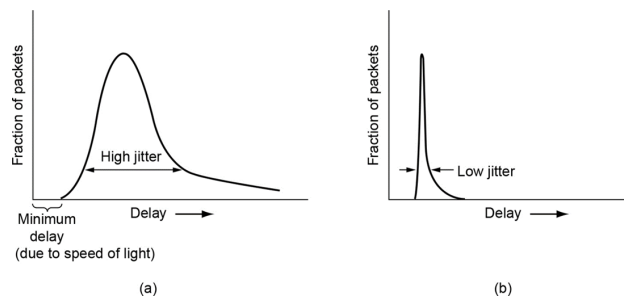
- **Version (V)** 2 bit
Indica la versione del protocollo RTP. Attualmente, il valore è fissato a 2.

- **Padding (P)** 1 bit
Se impostato a 1, il pacchetto contiene byte di padding extra in coda, utilizzati per allineamento. L'ultimo byte del pacchetto indica la quantità di padding.
- **Extension (X)** 1 bit
Se impostato a 1, è presente un'intestazione estesa opzionale, la cui struttura è definita dall'applicazione.
- **CSRC Count (CC)** 4 bit
Numero di identificatori CSRC presenti nell'header. Se il valore è 0, non ci sono contributori elencati.
- **Marker (M)** 1 bit
Bit utilizzabile dall'applicazione per segnalare eventi rilevanti, come la fine di un frame o di un'unità logica di dati.
- **Payload Type** 7 bit
Indica il tipo di dato trasportato (ad esempio, codec audio o video) secondo una codifica predefinita o dinamica.
- **Sequence Number** 16 bit
Numero di sequenza del pacchetto, incrementato di uno per ogni pacchetto trasmesso. Utilizzato per il rilevamento di perdite e per il riordinamento dei pacchetti.
- **Timestamp** 32 bit
Valore che rappresenta l'istante di generazione del primo campione contenuto nel pacchetto, espresso secondo il clock specifico del flusso (ad esempio, 90 kHz per video).
- **Synchronization Source Identifier (SSRC)** 32 bit
Identificatore univoco della sorgente del flusso, scelto casualmente dal mittente per distinguere i flussi tra loro.
- **Contributing Source Identifiers (CSRC)** variabile
Elenco opzionale di identificatori di sorgenti che hanno contribuito al flusso, utilizzato ad esempio da mixer audio o bridge. Ogni CSRC occupa 32 bit e il numero totale è specificato dal campo CC.

4.3.4.2 Jitter

Il **Jitter** misura la variazione nel tempo di arrivo dei pacchetti su una rete. In altre parole, indica quanto i pacchetti arrivano con ritardi irregolari rispetto al tempo medio atteso.

Se i pacchetti dovessero arrivare sempre a intervalli regolari, il jitter sarebbe pari a zero. Un valore di jitter elevato può causare problemi in applicazioni sensibili al ritardo, come VoIP o video streaming.



4.3.4.3 RTCP

Il protocollo RTCP (Real-time Transport Control Protocol) è progettato per operare in parallelo con RTP, completandone le funzionalità. A differenza di RTP, RTCP non trasporta dati multimediali, ma fornisce funzioni di controllo, retroazione e sincronizzazione essenziali per le comunicazioni in tempo reale.

Le principali funzionalità offerte da RTCP sono:

- **Retroazione (Feedback):** RTCP trasmette periodicamente pacchetti di controllo che informano il mittente sulla qualità della trasmissione, come tassi di perdita, jitter, latenza o congestione. Queste

informazioni consentono al mittente di adattare dinamicamente parametri come il bitrate o il tipo di codifica, migliorando la qualità percepita.

- **Sincronizzazione:** RTCP consente la sincronizzazione tra flussi multimediali distinti appartenenti alla stessa sessione (ad esempio, audio e video).
- **Identificazione e Interfaccia utente:** I pacchetti RTCP trasportano informazioni testuali come nomi dei partecipanti o identificatori, migliorando la gestione delle sessioni e fornendo interfacce utente più informative.

4.4 TCP

Il protocollo TCP (Transmission Control Protocol) nasce per fornire un servizio di comunicazione affidabile, orientato alla connessione e full-duplex tra due host su una rete non affidabile come Internet. TCP garantisce la consegna ordinata e priva di errori di un flusso di byte tra sorgente e destinazione, indipendentemente dal numero di reti o router attraversati.

Sebbene i dati vengano trasmessi fisicamente in pacchetti chiamati **segmenti**, TCP maschera questa suddivisione, presentando alle applicazioni un flusso continuo di byte, senza vincoli di dimensione e senza visibilità sulla segmentazione.

L'identificazione delle estremità della comunicazione avviene tramite il concetto di **socket**.

4.4.1 Socket

Un **socket** è definito come una coppia ordinata:

< IP address, Port number >

Il numero di porta è un intero a 16 bit, quindi compreso tra 0 e 65535. Le porte TCP si suddividono in tre categorie:

- **Well-Known Ports** (0 - 1023): riservate a servizi standard (es. HTTP porta 80, HTTPS porta 443, SSH porta 22).
- **Registered Ports** (1024 - 49151): possono essere utilizzate da applicazioni comuni, spesso assegnate ufficialmente su richiesta.
- **Dynamic o Ephemeral Ports** (49152 - 65535): assegnate dinamicamente dal sistema operativo per le comunicazioni temporanee.

4.4.2 Servizi

Il protocollo TCP fornisce i seguenti servizi:

- **Multiplexing e demultiplexing** basati su numeri di porta per distinguere le diverse connessioni.
- **Rilevamento degli errori** tramite checksum sull'intero segmento.
- **Consegna ordinata** dei dati, mantenendo l'ordine corretto dei byte.
- **Recupero delle perdite** (*loss recovery*) tramite ritrasmissioni.
- **Gestione del flusso** attraverso finestre scorrevoli per evitare che il ricevente venga sovraccaricato.
- **Gestione della congestione** per adattare dinamicamente la trasmissione alle condizioni della rete.

4.4.3 ACK

Il protocollo TCP implementa meccanismi di conferma per garantire l'affidabilità della comunicazione. Ogni segmento inviato deve essere confermato dal destinatario tramite un messaggio di **ACK** (Acknowledgment), ad eccezione dei casi in cui viene applicata la tecnica del **Delayed ACK**, che consente di ritardare l'invio della conferma fino a un massimo di due segmenti consecutivi, riducendo il traffico di controllo.

TCP utilizza un meccanismo di **ACK cumulativo**: l'ACK inviato dal destinatario contiene il **numero di sequenza** del prossimo byte che si aspetta di ricevere, che implica implicitamente la conferma della corretta ricezione di tutti i byte precedenti.

4.4.3.1 Gestione degli errori

Nel caso in cui un segmento venga perso o corrotto, il destinatario, alla ricezione di segmenti successivi fuori ordine, invia un **Duplicate ACK (DupACK)**. Questo ACK contiene sempre lo stesso numero di sequenza relativo al primo byte mancante, segnalando al mittente la presenza di una perdita.

La gestione con dupACK è stata introdotta nel TCP a partire dalla versione **Tahoe**.

4.4.3.2 RTT

Il tempo che intercorre tra l'invio di un segmento e la ricezione del relativo ACK viene definito **RTT** (Round Trip Time).

4.4.4 Segmento

Ogni unità di trasmissione del protocollo TCP prende il nome di **segmento**. Ciascun segmento è costituito da un'intestazione (**header**) di lunghezza minima pari a 20 byte, che può essere estesa tramite campi opzionali, e da un campo dati (payload) contenente il flusso di byte da trasmettere.

Il campo più importante dell'header è il **numero di sequenza** (*Sequence Number*), un valore a 32 bit che identifica il primo byte del flusso trasportato dal segmento rispetto alla connessione TCP. Questo valore consente di garantire il corretto ordinamento dei dati e il rilevamento di duplicati o perdite.

4.4.4.1 Timeout

Ogni segmento inviato viene associato a un **timer** di ritrasmissione. Se il mittente non riceve un ACK valido entro il tempo prestabilito (**Timeout**), considera il segmento perso e lo reinvia.

4.4.5 Header

Ogni segmento TCP inizia con un'intestazione (**header**) di lunghezza minima pari a 20 byte, estendibile attraverso il campo opzionale **Options**. A seguire, possono essere presenti fino a circa 65.535 byte di dati, in base alla dimensione massima del pacchetto IP.

La struttura dettagliata dell'header TCP è la seguente:

- **Source Port** 16 bit
Numero di porta del processo mittente.
- **Destination Port** 16 bit
Numero di porta del processo destinatario.
- **Sequence Number** 32 bit
Numero di sequenza del primo byte di dati trasportato nel segmento.
- **Acknowledgment Number** 32 bit
Numero di sequenza del prossimo byte atteso dal mittente del segmento.
- **Header Length** 4 bit
Lunghezza dell'header TCP in multipli di 4 byte.
- **Riservato** 3 bit
Spazio riservato per utilizzi futuri, attualmente posto a zero.
- **Flags** 8 bit
Bit di controllo che definiscono lo stato e il comportamento della connessione. I flag più comuni sono:
 - **ECE** (ECN-Echo)
Indica che il mittente ha rilevato una notifica esplicita di congestione (ECN) dalla rete. In risposta, il destinatario può adattare la trasmissione.
 - **CWR** (Congestion Window Reduced)
Segnalato dal mittente per indicare di aver ridotto la finestra di congestione, in risposta a un segnale ECN.

- **URG** (Urgent)
Indica che il campo *Urgent Pointer* è valido e identifica la fine dei dati urgenti nel flusso.
 - **ACK** (Acknowledgment)
Specifica che il campo *Acknowledgment Number* è valido e contiene il prossimo byte atteso.
 - **PSH** (Push)
Richiede la consegna immediata dei dati all'applicazione, senza ulteriori attese o bufferizzazione da parte della pila TCP.
 - **RST** (Reset)
Richiede il reset immediato della connessione.
 - **SYN** (Synchronize)
Utilizzato per iniziare una connessione e sincronizzare i numeri di sequenza.
 - **FIN** (Finish)
Segnala che il mittente ha terminato l'invio dei dati e desidera chiudere la connessione in modo ordinato.
- **Window Size** 16 bit
Numero di byte inviabili a partire da dall'ACK.
 - **Checksum** 16 bit
Verifica di integrità che copre header, dati e pseudo-header IP. A differenza dell'UDP, il numero di protocollo vale 6 e il campo è obbligatorio.
 - **Urgent Pointer** 16 bit
Indice che indica la posizione dei dati urgenti nel flusso.
 - **Options** variabile
Campi opzionali per funzionalità avanzate.

4.4.5.1 Pseudo Header

Il calcolo del checksum TCP utilizza, oltre ai dati del segmento, anche uno **pseudo header** che non viene trasmesso ma è incluso nel calcolo per proteggere le informazioni fondamentali a livello IP.

Lo pseudo header TCP contiene:

- **Source Address** 32 bit
Indirizzo IP sorgente.
- **Destination Address** 32 bit
Indirizzo IP di destinazione.
- **Protocol** 8 bit
Valore fisso 6 per TCP.
- **TCP Segment Length** 16 bit
Lunghezza totale del segmento TCP (header + dati).

4.4.6 Connessione

Per stabilire una connessione TCP si segue il seguente processo:

- Un lato (solitamente il server) si pone in ascolto in modo passivo, eseguendo nell'ordine le primitive **LISTEN** e **ACCEPT**, che possono essere configurate per accettare connessioni da una sorgente specifica o da qualsiasi mittente.
- L'altro lato (ad esempio il client) esegue la primitiva **CONNECT**, specificando:
 - L'indirizzo IP e la porta della destinazione.
 - La dimensione massima del segmento TCP che è disposto ad accettare.
 - Facoltativamente, alcuni dati utente.
- La primitiva **CONNECT** invia un segmento TCP con il flag **SYN** impostato a 1 e il flag **ACK** a 0, iniziando la procedura di connessione.
- Quando il server riceve questo segmento:

- Controlla se esiste un processo in ascolto sulla porta specificata nel campo **Destination Port**.
- Se nessun processo è in ascolto, il server risponde con un segmento TCP con il flag **RST** (Reset) a 1, rifiutando la connessione.
- Se esiste un processo in ascolto, il server può accettare la connessione, consegnando il segmento al processo applicativo.
- Se il server accetta la connessione:
 - Risponde con un segmento TCP contenente i flag **SYN** e **ACK**.
- Infine, il client risponde con un segmento contenente il flag **ACK**, completando il processo di connessione noto come **Three-Way Handshake**.

4.4.6.1 Connessione Simultanea

Nel caso in cui due host tentino di stabilire una connessione TCP tra loro esattamente nello stesso momento, si verifica una situazione detta **connessione simultanea**.

Entrambi gli host inviano un segmento con flag **SYN** all'altro, specificando lo stesso insieme di endpoint:

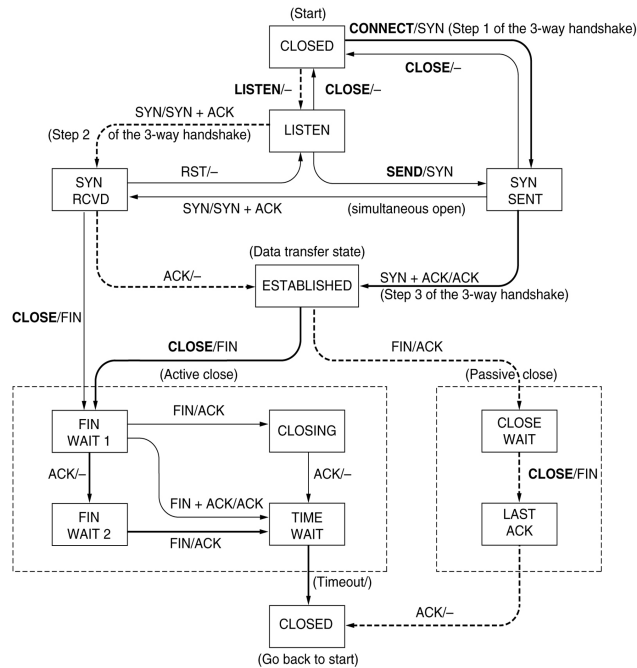
< IP Sorgente, Porta Sorgente, IP Destinazione, Porta Destinazione >

Poiché le connessioni TCP sono identificate univocamente da questa quaterna di valori, il risultato finale è comunque la creazione di un'unica connessione valida tra i due host, evitando duplicazioni o conflitti. Entrambi gli host completano il *Three-Way Handshake* incrociato e la comunicazione può proseguire normalmente.

4.4.6.2 Stati di Connessione

Una connessione TCP può trovarsi in uno dei seguenti 11 stati, che riflettono l'avanzamento del processo di instaurazione, utilizzo e chiusura della connessione:

- **CLOSED**: nessuna connessione attiva o in corso.
- **LISTEN**: il server è in attesa di richieste di connessione in ingresso. Stato tipico dopo una chiamata a `listen()`.
- **SYN SENT**: il client ha avviato una richiesta di connessione attiva inviando un segmento **SYN** (tramite `connect()`).
- **SYN RECEIVED**: è stato ricevuto un **SYN** e si è risposto con un **SYN-ACK**; in attesa della conferma finale dal client.
- **ESTABLISHED**: la connessione è attiva, entrambe le parti possono scambiarsi dati. Questo è lo stato operativo della connessione.
- **FIN WAIT 1**: un lato ha richiesto la chiusura della connessione inviando un **FIN**, ma non ha ancora ricevuto conferma.
- **FIN WAIT 2**: è stata ricevuta la conferma della chiusura dall'altra parte, si attende il **FIN** finale.
- **CLOSE WAIT**: il lato locale ha ricevuto un **FIN** dall'altra parte e deve ancora chiudere la connessione.
- **CLOSING**: entrambi i lati hanno inviato un **FIN** quasi contemporaneamente, si attende la conferma.
- **LAST ACK**: è stato inviato un **FIN** finale, si attende l'ultimo **ACK** prima di chiudere.
- **TIME WAIT**: la connessione è tecnicamente chiusa, ma si attende un periodo definito (tipicamente $2 \times \text{Maximum Segment Lifetime}$) per assicurarsi che eventuali pacchetti persi o ritardati non interferiscano.

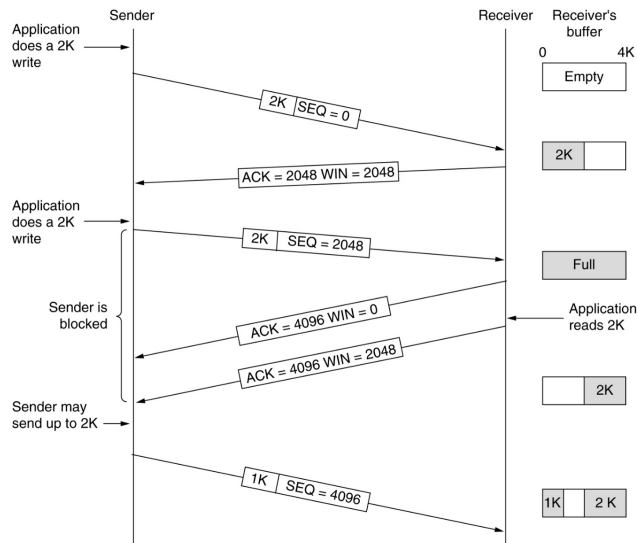


La linea **spessa continua** rappresenta il percorso standard degli host.
 La linea **spessa tratteggiata** rappresenta il percorso standard dei server.
 La linea **sottile** rappresenta percorsi non standard.

4.5 Controllo di Flusso

Il controllo di flusso in TCP si basa sul meccanismo delle **finestre scorrevoli**, che permette di evitare che il mittente sovraccarichi il destinatario inviando più dati di quanti ne possa effettivamente gestire.

Il campo **Window Size**, presente nell'header TCP, indica la quantità di byte che il destinatario è attualmente in grado di ricevere e memorizzare nei propri buffer. Questo valore è comunicato costantemente al mittente, che deve rispettarlo per regolare la velocità di trasmissione.



4.5.1 Finestra di Ricezione

- Se la **Window Size** vale 0, il destinatario segnala al mittente l'impossibilità temporanea di ricevere ulteriori dati, con l'eccezione dei segmenti contrassegnati come **urgenti**.
- Quando la finestra è chiusa ($\text{Window Size} = 0$), il mittente può inviare periodicamente un **Window Probe**: un piccolo segmento che serve a verificare se il destinatario ha nuovamente spazio disponibile.

4.5.2 Timeout e Ritrasmissione

Dopo l'invio di ogni segmento, il protocollo TCP avvia un **timer di ritrasmissione**. Se il timer scade prima della ricezione dell'**ACK** corrispondente, si presume che il segmento sia andato perso o che vi sia congestione nella rete, pertanto il segmento viene ritrasmesso.

Questa procedura è nota come **RTO** (Retransmission Timeout).

4.5.2.1 Calcolo del Timer

Il valore del timer viene calcolato dinamicamente in base al **Round Trip Time (RTT)** stimato, ovvero il tempo medio necessario affinché un segmento raggiunga il destinatario e l'**ACK** ritorni al mittente.

4.5.2.2 Gestione dei Ritrasmissioni

Quando un segmento viene ritrasmesso a causa dello scadere del timer, TCP applica il meccanismo di **backoff esponenziale**: il valore del timer viene raddoppiato progressivamente a ogni nuovo tentativo, fino a un massimo predefinito. Questo accorgimento evita di saturare ulteriormente la rete in situazioni di congestione.

4.5.2.3 Delayed ACK

Per ottimizzare il traffico e ridurre il numero di pacchetti in rete, il destinatario può ritardare intenzionalmente l'invio degli **ACK (Delayed ACK)**, aspettando di ricevere più segmenti prima di inviare una singola conferma cumulativa. Tuttavia, il ritardo non deve mai superare un intervallo massimo predefinito, proprio per non interferire con il timer di ritrasmissione del mittente.

4.5.2.4 Selective ACK

TCP supporta un meccanismo chiamato **SACK** (Selective Acknowledgment), che permette di segnalare la ricezione di segmenti non consecutivi.

Quando il destinatario riceve pacchetti fuori ordine, oltre al normale **ACK** per l'ultimo byte ricevuto in ordine, invia anche un **SACK** che specifica i blocchi di dati ricevuti correttamente.

Esempio: se vengono inviati segmenti numerati da 1 a 5 e il destinatario riceve i segmenti 1 e 3 (ma non il 2), invierà:

- Un **ACK** per il segmento 1 (ultimo in ordine ricevuto).
- Un **SACK** che indica di aver ricevuto anche il segmento 3.

Questo meccanismo accelera il rilevamento e la ritrasmissione dei pacchetti mancanti, migliorando l'efficienza della comunicazione.

4.6 Controllo di Congestione

Nel contesto di Internet, si parla di **congestione** quando le code dei router si saturano, costringendo questi ultimi a scartare pacchetti in ingresso.

Il protocollo TCP implementa un meccanismo di controllo della congestione basato su una finestra dinamica, detta **congestion window** (*cwnd*). Questo meccanismo interpreta la perdita di pacchetti come un segnale implicito di congestione sulla rete.

4.6.1 Finestra di Congestione

La finestra di congestione (*cwnd*) rappresenta il numero massimo di byte che il mittente può avere "in volo" sulla rete, ovvero trasmessi ma non ancora confermati da ACK.

La velocità di trasmissione effettiva è espressa da:

$$T_x = \frac{cwnd}{RTT}$$

dove:

- *cwnd* è la dimensione della finestra di congestione, misurata in byte.
- *RTT* è il tempo stimato di andata e ritorno della connessione.

4.6.2 Segnali di Congestione

TCP interpreta la perdita di pacchetti come un segnale binario di congestione sulla rete. I principali indicatori sono:

- **Timeout di ritrasmissione (RTO)**: se un ACK non viene ricevuto entro il tempo previsto, si presume congestione o perdita di pacchetti.
- **ACK duplicati multipli (DupACK)**: la ricezione di tre o più ACK duplicati indica probabile perdita di un pacchetto, ma la rete è ancora operativa.

Attraverso questi meccanismi, TCP monitora lo stato della rete e regola dinamicamente la dimensione della finestra di congestione.

4.6.2.1 ACK Clock

TCP utilizza un meccanismo chiamato **ACK clock** per regolare il flusso di dati e prevenire la congestione della rete. Questo meccanismo si basa sull'idea che la ricezione degli ACK da parte del mittente riflette la capacità effettiva della rete nel trasportare dati.

In pratica:

- Il mittente invia un certo numero di segmenti.
- Ogni segmento che raggiunge correttamente il destinatario genera un ACK di ritorno.
- Il ritmo con cui gli ACK arrivano al mittente viene utilizzato come "orologio naturale" per regolare la trasmissione successiva.

In questo modo, il mittente adatta dinamicamente la propria velocità di invio sulla base della capacità del percorso end-to-end, evitando di saturare i buffer dei router o dei nodi intermedi. Se la rete è lenta o congestionata, gli ACK arrivano più distanziati, inducendo il mittente a rallentare; se la rete è libera, gli ACK arrivano più rapidamente, consentendo un aumento progressivo della velocità di trasmissione.

L'ACK clock è fondamentale per garantire una trasmissione sostenibile e stabile, specialmente su reti con capacità variabili o in presenza di congestione.

4.6.3 Slow Start

L'algoritmo di **Slow Start** regola l'inizio della trasmissione dati in una nuova connessione TCP, consentendo un aumento progressivo della velocità di invio per evitare congestione improvvisa della rete.

Il funzionamento è il seguente:

1. All'inizio, la finestra di congestione (*cwnd*) è inizializzata solitamente a 1 o 2 segmenti MSS (Maximum Segment Size).
2. Per ogni ACK ricevuto, *cwnd* viene incrementata di 1 MSS. Poiché ogni segmento inviato genera un ACK, la crescita è esponenziale: la finestra raddoppia ogni RTT (Round Trip Time) se tutti i segmenti arrivano correttamente.

3. In altre parole, il mittente segue l'**ACK clock**, sfruttando il ritmo degli ACK per dosare la trasmissione:
 - Per ogni ACK ricevuto, il mittente può inviare un nuovo segmento per il posto liberato e un ulteriore segmento per l'aumento di finestra.
4. Per prevenire congestione eccessiva, viene introdotta una soglia chiamata **Slow Start Threshold** (**ssthresh**), che limita la crescita esponenziale:
 - Se **cwnd** **ssthresh**, il mittente opera in modalità Slow Start (crescita esponenziale).
 - Se **cwnd** $\geq$ **ssthresh**, il mittente passa alla fase di **Congestion Avoidance** (crescita lineare).
5. Alla rilevazione di una perdita (timeout o triplo DupACK), si interpreta come segnale di congestione:
 - **ssthresh** viene dimezzata.
 - **cwnd** viene riportata a 1 MSS e si riparte con una nuova fase di Slow Start.

Questo meccanismo consente a TCP di adattarsi gradualmente alla capacità della rete, evitando congestioni immediate e migliorando la stabilità della trasmissione.

4.6.4 Congestion Avoidance

Quando la dimensione della finestra di congestione (**cwnd**) supera la soglia **ssthresh** (Slow Start Threshold), il protocollo TCP passa dalla fase di **Slow Start** alla modalità di **Congestion Avoidance**. In questa fase, la crescita della finestra avviene in maniera più controllata e lineare per evitare congestioni:

- Per ogni **ACK** ricevuto, la finestra **cwnd** viene aumentata di:

$$\Delta cwnd = \frac{MSS^2}{cwnd}$$

dove:

- **MSS** è il **Maximum Segment Size**, ovvero la dimensione massima di un singolo segmento.
- **cwnd** è la finestra di congestione attuale, espressa in byte.
- Dato che in un RTT vengono inviati circa $\frac{cwnd}{MSS}$ segmenti, la crescita complessiva della finestra risulta pari a circa 1 MSS per ogni RTT, realizzando quindi un incremento lineare.

La perdita di pacchetti viene interpretata come segnale di congestione, rilevabile in due modi:

- **Timeout di ritrasmissione (RTO):**
 - Si presume congestione grave.
 - **ssthresh** viene impostato a metà del valore corrente di **cwnd**.
 - **cwnd** viene ridotta a 1 MSS e il mittente torna in fase di Slow Start.
- **Triplo DupACK** (tre o più ACK duplicati):
 - Si interpreta come perdita isolata di un segmento, ma la rete non è completamente congestionata.
 - Si entra nella fase di **Fast Retransmission**, evitando un completo reset della finestra.

In questo modo, TCP regola dinamicamente la trasmissione per adattarsi alle condizioni della rete, bilanciando efficienza e affidabilità.

4.6.5 Fast Retransmission (TCP Tahoe)

L'algoritmo di **Fast Retransmission** consente al mittente TCP di rilevare la perdita di un segmento senza dover attendere lo scadere del timer di ritrasmissione (**RTO**).

Quando il mittente riceve **3 ACK duplicati (DupACK)**, interpreta la situazione come una probabile perdita di un singolo segmento.

Il comportamento del protocollo TCP in questa situazione è il seguente:

1. Viene immediatamente ritrasmesso il segmento perso (senza attendere lo scadere del timer RTO).
2. La soglia di Slow Start (**ssthresh**) viene ridotta a metà del valore attuale della finestra di congestione (**cwnd**).
3. **cwnd** viene impostata a 1 MSS.
4. Il protocollo riparte dalla fase di **Slow Start**.

Questo approccio consente di reagire più rapidamente alle perdite, migliorando la velocità di recupero, ma in TCP Tahoe ogni perdita porta comunque al completo azzeramento della finestra, con conseguente ripartenza graduale tramite Slow Start.

4.6.6 Fast Recovery (TCP Reno)

Il **Fast Recovery** è una modalità temporanea attivata dal protocollo TCP alla ricezione di **3 DupACK**, che permette di reagire alla perdita di un pacchetto senza dover ridurre drasticamente la velocità di trasmissione, mantenendo l'**ACK clock** attivo.

Il funzionamento avviene come segue:

1. Alla ricezione di 3 DupACK, il mittente rileva una probabile perdita e attiva la **Fast Retransmission** ritrasmettendo subito il segmento perso.
2. La soglia di Slow Start (**ssthresh**) viene aggiornata a metà dell'attuale valore della finestra di congestione (**cwnd**).
3. La finestra di congestione **cwnd** viene temporaneamente ridotta a un valore pari alla nuova **ssthresh**.
4. Si entra in modalità **Fast Recovery**, durante la quale:
 - L'**ACK clock** continua a funzionare per evitare che il flusso di dati si blocchi completamente.
 - Per ogni DupACK ricevuto (inclusi i tre iniziali), il mittente può trasmettere un nuovo segmento, mantenendo il canale occupato e sfruttando al massimo la capacità della rete.
5. Questa fase dura fino a quando non si riceve l'ACK del pacchetto ritrasmesso. Tale evento avviene circa dopo un RTT dal momento della Fast Retransmission.
6. Alla ricezione dell'ACK del pacchetto perso:
 - Il flusso di DupACK si interrompe.
 - La modalità Fast Recovery termina.
 - La finestra di congestione viene impostata al valore di **ssthresh** e inizia ad aumentare secondo la crescita lineare prevista dalla fase di **Congestion Avoidance**.

Questo approccio permette a TCP di reagire più rapidamente a perdite isolate, evitando una completa ripartenza da Slow Start come in TCP Tahoe e riducendo l'impatto della congestione sul flusso di trasmissione.

4.7 Velocità di Trasmissione

Si definisce con W il numero massimo di segmenti che possono essere inviati senza attendere ACK, ovvero la **finestra di congestione** o finestra di trasmissione effettiva.

Il tempo necessario per trasmettere W segmenti e ricevere le relative conferme dipende dal valore di W :

$$T_x = \begin{cases} RTT & \text{se } W = 1 \quad (\text{invio di un solo segmento e attesa dell'ACK}) \\ \frac{W}{RTT} & \text{se } W > 1 \quad (\text{invio continuo di } W \text{ segmenti, attesa complessiva degli ACK}) \end{cases}$$

Per saturare la banda disponibile della rete, è necessario che la velocità di trasmissione T_x sia pari al prodotto banda-ritardo:

$$W = B \times RTT$$

dove:

- W è il numero di byte o segmenti inviabili senza attendere ACK (**window size**).
- B è la banda disponibile (byte/sec).
- RTT è il Round Trip Time, ovvero il tempo medio di andata e ritorno di un pacchetto.

Tuttavia, il valore effettivo della finestra W è determinato dal minimo tra due fattori:

$$W = \min(cwnd, rwnd)$$

dove:

- $cwnd$ è la **congestion window**, imposta dal protocollo TCP in base allo stato di congestione della rete.
- $rwnd$ è la **receiver window**, ovvero la dimensione della finestra di ricezione comunicata dal destinatario, che riflette la capacità del buffer.

Ne consegue che la velocità massima di trasmissione T_x è:

$$T_x = \min\left(\frac{cwnd}{RTT}, \frac{rwnd}{RTT}\right)$$

dove:

- $\frac{cwnd}{RTT}$ rappresenta la massima velocità consentita dal controllo della congestione.
- $\frac{rwnd}{RTT}$ rappresenta la massima velocità consentita dal controllo di flusso.

TCP quindi regola dinamicamente la velocità di trasmissione per adattarsi sia alle condizioni della rete (congestione) sia alla capacità del destinatario.

4.7.1 Perdite

Un segmento viene ritenuto perso in due condizioni:

- Scade l'**RTO**: il mittente non ha ricevuto nessuno ACK che non fosse dupACK prima dello scadere il timer
- Arrivano 3 **dupACK**: al destinatario sono arrivati segmenti successivi a quelli che si aspettava